```
# -*- coding: utf-8 -*-
"""
RestoraMind 基于多模态推理与强化学习的图像复原智能体软件
软件将真实场景复合退化图像的复原建模为序列决策过程：
每一步由多模态智能体联合完成「退化类型推理 → 工具选择 → 工具执行」，
并以图像低层统计先验辅助同类工具之间的自适应选择，
通过强化学习（组相对策略优化 GRPO）端到端训练智能体策略；步级奖励也可供 PPO 等算法使用。
子模块：
    data        退化合成与训练数据集构建
    priors      图像低层统计先验提取
    tools       复原工具箱（本地算子 / 远程专家模型服务客户端）
    tool_server 复原工具服务端（HTTP 接口，统一封装专家模型）
    agent       智能体环境、提示词、策略
    reward      格式奖励、图像质量奖励、退化推理奖励、复合奖励
    train       GRPO 训练器与 VeRL 框架桥接
    inference   单图复原与数据集评估
"""
__version__ = "1.0"
__software_name__ = "RestoraMind 基于多模态推理与强化学习的图像复原智能体软件"
__copyright_holder__ = "深圳理工大学 邹翔宇、马智恒"
# -*- coding: utf-8 -*-
"""退化合成与数据集构建。"""
from .degradations import DEGRADATION_TYPES, DEGRADATION_LEVELS, apply_degradation
from .synthesize import DegradationSpec, synthesize_sample, SynthesizedSample
from .dataset import build_parquet_dataset, load_parquet_samples, Sample
__all__ = [
    "DEGRADATION_TYPES", "DEGRADATION_LEVELS", "apply_degradation",
    "DegradationSpec", "synthesize_sample", "SynthesizedSample",
    "build_parquet_dataset", "load_parquet_samples", "Sample",
]
# -*- coding: utf-8 -*-
"""
八类图像退化算子。
每个算子签名统一为 ``fn(img: np.ndarray[H,W,3] uint8 RGB, level: str, rng) -> np.ndarray``，
``level`` 取值 low / medium / high，对应退化强度区间由 ``LEVEL_PARAMS`` 给出。
算子之间可任意组合、任意顺序叠加，用于模拟真实场景的未知复合退化。
"""
from __future__ import annotations
from typing import Callable, Dict, Tuple
import cv2
import numpy as np
# 退化类型名称与智能体提示词、奖励模块中的名称保持一致
DEGRADATION_TYPES = (
    "haze",
    "rain",
    "defocus blur",
    "low resolution",
    "motion blur",
    "dark",
    "jpeg compression artifact",
```

```
    "noise",
)
DEGRADATION_LEVELS = ("low", "medium", "high")
# 每类退化在三个强度等级下的参数采样区间
LEVEL_PARAMS: Dict[str, Dict[str, Tuple[float, float]]] = {
    "haze": {"low": (0.2, 0.4), "medium": (0.4, 0.6), "high": (0.6, 0.85)},          # 透射率衰减
    "rain": {"low": (200, 400), "medium": (400, 800), "high": (800, 1500)},          # 雨线数量
    "defocus blur": {"low": (1.0, 2.0), "medium": (2.0, 3.5), "high": (3.5, 5.0)},   # 高斯 sigma
    "low resolution": {"low": (1.5, 2.0), "medium": (2.0, 3.0), "high": (3.0, 4.0)}, # 下采样倍率
    "motion blur": {"low": (5, 9), "medium": (9, 15), "high": (15, 25)},             # 运动核长度
    "dark": {"low": (1.5, 2.0), "medium": (2.0, 3.0), "high": (3.0, 4.5)},           # gamma
    "jpeg compression artifact": {"low": (40, 60), "medium": (20, 40), "high": (5, 20)},  # JPEG 质量
    "noise": {"low": (5, 15), "medium": (15, 30), "high": (30, 50)},                 # 高斯噪声 sigma
}
def _sample(rng: np.random.Generator, dtype: str, level: str) -> float:
    lo, hi = LEVEL_PARAMS[dtype][level]
    return float(rng.uniform(lo, hi))
def add_haze(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """大气散射模型 I = J * t + A * (1 - t) , t 为透射率。"""
    beta = _sample(rng, "haze", level)
    h, w = img.shape[:2]
    # 用从上到下的深度渐变模拟远景更浓的雾
    depth = np.linspace(0.6, 1.0, h, dtype=np.float32)[:, None]
    depth = np.repeat(depth, w, axis=1)
    t = np.exp(-beta * depth)[..., None]
    airlight = float(rng.uniform(0.8, 1.0)) * 255.0
    out = img.astype(np.float32) * t + airlight * (1.0 - t)
    return np.clip(out, 0, 255).astype(np.uint8)
def add_rain(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """绘制随机倾斜雨线并做轻微运动模糊。"""
    n = int(_sample(rng, "rain", level))
    h, w = img.shape[:2]
    layer = np.zeros((h, w), dtype=np.float32)
    angle = float(rng.uniform(-15, 15))
    length = int(rng.integers(8, 20))
    dx = int(round(length * np.sin(np.deg2rad(angle))))
    dy = int(round(length * np.cos(np.deg2rad(angle))))
    for _ in range(n):
        x, y = int(rng.integers(0, w)), int(rng.integers(0, h))
        cv2.line(layer, (x, y), (x + dx, y + dy), float(rng.uniform(0.4, 1.0)), 1)
    layer = cv2.GaussianBlur(layer, (3, 3), 0)
    out = img.astype(np.float32) + layer[..., None] * 200.0
    return np.clip(out, 0, 255).astype(np.uint8)
def add_defocus_blur(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """圆盘核近似的离焦模糊。"""
    sigma = _sample(rng, "defocus blur", level)
    radius = max(1, int(round(sigma * 1.5)))
    k = 2 * radius + 1
    kernel = np.zeros((k, k), dtype=np.float32)
    cv2.circle(kernel, (radius, radius), radius, 1.0, -1)
```

```
    kernel /= kernel.sum()
    return cv2.filter2D(img, -1, kernel)
def add_low_resolution(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """下采样后再上采样回原尺寸，模拟分辨率损失（尺寸保持不变以便指标计算）。"""
    scale = _sample(rng, "low resolution", level)
    h, w = img.shape[:2]
    small = cv2.resize(img, (max(8, int(w / scale)), max(8, int(h / scale))), interpolation=cv2.INTER_AREA)
    return cv2.resize(small, (w, h), interpolation=cv2.INTER_CUBIC)
def add_motion_blur(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """随机方向的线性运动模糊核。"""
    length = int(_sample(rng, "motion blur", level))
    angle = float(rng.uniform(0, 180))
    kernel = np.zeros((length, length), dtype=np.float32)
    kernel[length // 2, :] = 1.0
    rot = cv2.getRotationMatrix2D((length / 2 - 0.5, length / 2 - 0.5), angle, 1.0)
    kernel = cv2.warpAffine(kernel, rot, (length, length))
    kernel /= max(kernel.sum(), 1e-6)
    return cv2.filter2D(img, -1, kernel)
def add_dark(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """gamma 压暗并叠加轻微泊松噪声模拟低光成像。"""
    gamma = _sample(rng, "dark", level)
    x = img.astype(np.float32) / 255.0
    out = np.power(x, gamma) * 255.0
    out = out + rng.normal(0, 2.0, size=out.shape).astype(np.float32)
    return np.clip(out, 0, 255).astype(np.uint8)
def add_jpeg(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """低质量 JPEG 编解码产生块效应与振铃伪影。"""
    quality = int(_sample(rng, "jpeg compression artifact", level))
    ok, buf = cv2.imencode(".jpg", cv2.cvtColor(img, cv2.COLOR_RGB2BGR), [cv2.IMWRITE_JPEG_QUALITY, quality])
    if not ok:
        return img
    dec = cv2.imdecode(buf, cv2.IMREAD_COLOR)
    return cv2.cvtColor(dec, cv2.COLOR_BGR2RGB)
def add_noise(img: np.ndarray, level: str, rng: np.random.Generator) -> np.ndarray:
    """加性高斯白噪声。"""
    sigma = _sample(rng, "noise", level)
    out = img.astype(np.float32) + rng.normal(0, sigma, size=img.shape).astype(np.float32)
    return np.clip(out, 0, 255).astype(np.uint8)
DEGRADATION_FUNCTIONS: Dict[str, Callable[[np.ndarray, str, np.random.Generator], np.ndarray]] = {
    "haze": add_haze,
    "rain": add_rain,
    "defocus blur": add_defocus_blur,
    "low resolution": add_low_resolution,
    "motion blur": add_motion_blur,
    "dark": add_dark,
    "jpeg compression artifact": add_jpeg,
    "noise": add_noise,
}
def apply_degradation(img: np.ndarray, dtype: str, level: str, rng: np.random.Generator | None = None) -> np.nda
    rray:
```

```
    """对 RGB uint8 图像施加一种指定类型与强度的退化。"""
    if dtype not in DEGRADATION_FUNCTIONS:
        raise ValueError(f"未知退化类型: {dtype}，可选: {DEGRADATION_TYPES}")
    if level not in DEGRADATION_LEVELS:
        raise ValueError(f"未知退化强度: {level}，可选: {DEGRADATION_LEVELS}")
    rng = rng if rng is not None else np.random.default_rng()
    return DEGRADATION_FUNCTIONS[dtype](img, level, rng)
# -*- coding: utf-8 -*-
"""
复合退化合成：对干净图像随机施加 2~3 种退化（随机类型、随机强度、随机顺序），
并记录真实退化序列，作为训练阶段退化推理奖励的参考。
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import List, Optional, Sequence
import numpy as np
from .degradations import DEGRADATION_LEVELS, DEGRADATION_TYPES, apply_degradation
@dataclass
class DegradationSpec:
    """一次退化施加的记录：类型 + 强度。"""
    degradation_type: str
    degradation_level: str
    def to_dict(self) -> dict:
        """转换为可序列化的字典。"""
        return {"degradation_type": self.degradation_type, "degradation_level": self.degradation_level}
@dataclass
class SynthesizedSample:
    """合成样本：干净图、退化图、按施加顺序排列的退化序列。"""
    clean: np.ndarray
    degraded: np.ndarray
    degradations: List[DegradationSpec] = field(default_factory=list)
    @property
    def degradation_types(self) -> List[str]:
        """按施加顺序排列的退化类型。"""
        return [d.degradation_type for d in self.degradations]
    @property
    def env_name(self) -> str:
        """与训练数据 ``env_name`` 字段一致的逗号分隔退化类型串；无退化时为 clean。"""
        return ", ".join(self.degradation_types) if self.degradations else "clean"
def synthesize_sample(
    clean: np.ndarray,
    rng: Optional[np.random.Generator] = None,
    num_degradations: Sequence[int] = (2, 3),
    candidate_types: Sequence[str] = DEGRADATION_TYPES,
    clean_prob: float = 0.0,
) -> SynthesizedSample:
    """
    对单张干净图像随机合成复合退化。
    参数:
        clean: RGB uint8 图像
```

```
        rng: 随机数发生器，便于复现
        num_degradations: 退化数量候选集合，默认 2 或 3
        candidate_types: 可用退化类型
        clean_prob: 以该概率保留干净图像作为 clean 样本（训练智能体识别无需复原的情况）
    """
    rng = rng if rng is not None else np.random.default_rng()
    if rng.uniform() < clean_prob:
        return SynthesizedSample(clean=clean, degraded=clean.copy(), degradations=[])
    k = int(rng.choice(list(num_degradations)))
    k = min(k, len(candidate_types))
    chosen = rng.choice(list(candidate_types), size=k, replace=False)
    img = clean.copy()
    specs: List[DegradationSpec] = []
    for dtype in chosen:
        level = str(rng.choice(DEGRADATION_LEVELS))
        img = apply_degradation(img, str(dtype), level, rng)
        specs.append(DegradationSpec(str(dtype), level))
    return SynthesizedSample(clean=clean, degraded=img, degradations=specs)
# -*- coding: utf-8 -*-
"""
训练/评估数据集构建与读取（Parquet 格式）。
字段与 VeRL 智能体训练框架兼容：
    data_source   数据来源标识
    prompt        对话提示（system + user），user 内含 <image> 占位符
    images        [{"bytes": PNG 字节}] 退化图像
    env_name      逗号分隔的真实退化类型（施加顺序），clean 表示无退化
    reward_model  [{"degradation_type", "degradation_level"}, ...]
    extra_info    {"original_image": {"bytes": PNG}, "use_original": True, "index": i}
"""
from __future__ import annotations
import io
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterable, List, Optional, Sequence
import numpy as np
import pandas as pd
from PIL import Image
from .synthesize import DegradationSpec, SynthesizedSample, synthesize_sample
IMAGE_EXTS = (".png", ".jpg", ".jpeg", ".bmp", ".webp")
@dataclass
class Sample:
    """环境/评估使用的统一样本结构。"""
    degraded: Image.Image
    clean: Optional[Image.Image]
    degradations: List[DegradationSpec] = field(default_factory=list)
    index: int = 0
    @property
    def degradation_types(self) -> List[str]:
        """真实退化类型列表（施加顺序）。"""
        return [d.degradation_type for d in self.degradations]
```

```
    @property
    def is_clean(self) -> bool:
        """是否为无退化的干净样本。"""
        return len(self.degradations) == 0
def encode_png(img: np.ndarray | Image.Image) -> bytes:
    """把图像编码为 PNG 字节。"""
    pil = img if isinstance(img, Image.Image) else Image.fromarray(img)
    buf = io.BytesIO()
    pil.convert("RGB").save(buf, format="PNG")
    return buf.getvalue()
def decode_image(data) -> Image.Image:
    """兼容 {'bytes': ...} / bytes / 路径 / PIL 四种形式。"""
    if isinstance(data, Image.Image):
        return data.convert("RGB")
    if isinstance(data, dict):
        data = data.get("bytes")
    if isinstance(data, (bytes, bytearray)):
        return Image.open(io.BytesIO(data)).convert("RGB")
    if isinstance(data, str):
        return Image.open(data).convert("RGB")
    raise TypeError(f"无法解析的图像数据类型: {type(data)}")
def list_images(root: str | Path) -> List[Path]:
    """递归列出目录下的图像文件。"""
    root = Path(root)
    return sorted(p for p in root.rglob("*") if p.suffix.lower() in IMAGE_EXTS)
def load_clean_image(path: str | Path, max_size: int = 512) -> np.ndarray:
    """读取干净图像并限制最长边，避免训练时显存/内存压力。"""
    img = Image.open(path).convert("RGB")
    w, h = img.size
    scale = max(w, h) / float(max_size)
    if scale > 1.0:
        img = img.resize((max(8, int(w / scale)), max(8, int(h / scale))), Image.BICUBIC)
    return np.array(img)
def build_record(sample: SynthesizedSample, index: int, system_prompt: str, user_prompt: str,
                 data_source: str = "restoramind") -> dict:
    """把合成样本转换为一条 Parquet 记录。"""
    return {
        "data_source": data_source,
        "prompt": [
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
        "images": [{"bytes": encode_png(sample.degraded)}],
        "env_name": sample.env_name,
        "reward_model": [d.to_dict() for d in sample.degradations],
        "extra_info": {
            "original_image": {"bytes": encode_png(sample.clean)},
            "use_original": True,
            "index": index,
        },
```

```
    }
def build_parquet_dataset(
    clean_images: Iterable[str | Path | np.ndarray],
    output_path: str | Path,
    system_prompt: str,
    user_prompt_template: str,
    seed: int = 0,
    num_degradations: Sequence[int] = (2, 3),
    clean_prob: float = 0.0,
    max_size: int = 512,
    samples_per_image: int = 1,
) -> int:
    """
    从干净图像集合合成复合退化数据集并写入 Parquet。
    user_prompt_template 中可使用 {degradations} 占位符注入真实退化类型提示
    （训练时提供退化集合提示，智能体学习顺序与工具选择；评估时可关闭提示）。
    返回写入的样本数。
    """
    rng = np.random.default_rng(seed)
    records: List[dict] = []
    idx = 0
    for item in clean_images:
        clean = item if isinstance(item, np.ndarray) else load_clean_image(item, max_size=max_size)
        for _ in range(samples_per_image):
            syn = synthesize_sample(clean, rng, num_degradations=num_degradations, clean_prob=clean_prob)
            hint = ", ".join(syn.degradation_types) if syn.degradations else "none"
            user_prompt = user_prompt_template.format(degradations=hint)
            records.append(build_record(syn, idx, system_prompt, user_prompt))
            idx += 1
    df = pd.DataFrame(records)
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    df.to_parquet(output_path, index=False)
    return len(records)
def _as_list(value) -> list:
    """Parquet 读回的列表字段可能是 numpy 数组或 None，统一转为 list。"""
    if value is None:
        return []
    if isinstance(value, np.ndarray):
        return value.tolist()
    return list(value)
def record_to_sample(row: dict, index: int) -> Sample:
    """把 Parquet 记录解析为 Sample。"""
    degraded = decode_image(_as_list(row["images"])[0])
    extra = row.get("extra_info")
    extra = extra if isinstance(extra, dict) else {}
    clean = decode_image(extra["original_image"]) if extra.get("original_image") is not None else None
    specs = [DegradationSpec(d["degradation_type"], d.get("degradation_level", "medium"))
             for d in _as_list(row.get("reward_model")) if isinstance(d, dict)]
    return Sample(degraded=degraded, clean=clean, degradations=specs, index=index)
```

```
def load_parquet_samples(path: str | Path, num_samples: Optional[int] = None) -> List[Sample]:
    """读取 Parquet 数据集为 Sample 列表。"""
    df = pd.read_parquet(path)
    if num_samples is not None:
        df = df.head(num_samples)
    return [record_to_sample(row, i) for i, row in enumerate(df.to_dict("records"))]
# -*- coding: utf-8 -*-
"""图像低层统计先验。"""
from .image_statistics import ImageStatistics, compute_image_statistics, qualitative_level
__all__ = ["ImageStatistics", "compute_image_statistics", "qualitative_level"]
# -*- coding: utf-8 -*-
"""
图像低层统计先验提取。
在每个决策状态下，以极低计算代价（传统图像处理）提取当前图像的
颜色分布、亮度、对比度、边缘密度、频域能量以及退化估计特征，
组织为结构化文本随图像一起输入多模态智能体，用于
（1）辅助判断剩余退化类型；（2）在同一退化类型的多个候选工具之间做细粒度自适应选择。
"""
from __future__ import annotations
from dataclasses import asdict, dataclass
from typing import Dict, Tuple
import cv2
import numpy as np
from PIL import Image
# 各统计量的「低 / 中 / 高」分界阈值（经验值，可按数据分布调整）
LEVEL_THRESHOLDS: Dict[str, Tuple[float, float]] = {
    "brightness_mean": (70.0, 150.0),
    "dark_ratio": (0.15, 0.40),
    "contrast_global": (35.0, 60.0),
    "saturation_mean": (0.15, 0.35),
    "edge_density": (0.04, 0.12),
    "high_freq_energy": (0.05, 0.15),
    "noise_sigma": (4.0, 12.0),
    "blur_score": (50.0, 300.0),  # 拉普拉斯方差，越低越模糊；此项按反向解释
    "blockiness": (0.02, 0.06),
    "haze_score": (0.35, 0.55),
}
def qualitative_level(name: str, value: float) -> str:
    """将连续统计量映射为 low / medium / high 的定性描述。"""
    lo, hi = LEVEL_THRESHOLDS[name]
    if value < lo:
        return "low"
    if value < hi:
        return "medium"
    return "high"
@dataclass
class ImageStatistics:
    """单张图像的低层统计特征。"""
    width: int
    height: int
```

```
    brightness_mean: float      # 灰度均值 [0,255]
    dark_ratio: float           # 暗区像素比例（灰度 < 40）
    highlight_ratio: float      # 高光像素比例（灰度 > 220）
    contrast_global: float      # 灰度标准差
    saturation_mean: float      # HSV 饱和度均值 [0,1]
    color_cast: float           # 通道均值最大偏差 / 255，衡量色偏
    edge_density: float         # Canny 边缘像素比例
    high_freq_energy: float     # 高频频谱能量占比
    noise_sigma: float          # 噪声水平估计（中值滤波残差 MAD）
    blur_score: float           # 拉普拉斯方差，越小越模糊
    blockiness: float           # 8x8 块边界梯度不连续度，衡量 JPEG 块效应
    haze_score: float           # 暗通道均值 [0,1]，越高雾越浓
    def to_dict(self) -> dict:
        """转换为字典。"""
        return asdict(self)
    def to_qualitative(self) -> Dict[str, str]:
        """定性描述，供提示词使用。"""
        q = {
            "brightness": qualitative_level("brightness_mean", self.brightness_mean),
            "dark_region_ratio": qualitative_level("dark_ratio", self.dark_ratio),
            "contrast": qualitative_level("contrast_global", self.contrast_global),
            "saturation": qualitative_level("saturation_mean", self.saturation_mean),
            "edge_density": qualitative_level("edge_density", self.edge_density),
            "high_frequency_energy": qualitative_level("high_freq_energy", self.high_freq_energy),
            "noise_estimate": qualitative_level("noise_sigma", self.noise_sigma),
            "jpeg_blockiness": qualitative_level("blockiness", self.blockiness),
            "haze_estimate": qualitative_level("haze_score", self.haze_score),
        }
        # 模糊度反向：拉普拉斯方差低 → 模糊程度高
        inv = {"low": "high", "medium": "medium", "high": "low"}
        q["blur_estimate"] = inv[qualitative_level("blur_score", self.blur_score)]
        return q
    def to_prompt_text(self) -> str:
        """结构化文本，注入智能体的每个决策状态。"""
        q = self.to_qualitative()
        lines = [f"Image statistics ({self.width}x{self.height}):"]
        lines.append(f"- brightness: {q['brightness']} (mean={self.brightness_mean:.1f}, dark_ratio={self.dark_r
    atio:.2f})")
        lines.append(f"- contrast: {q['contrast']} (std={self.contrast_global:.1f})")
        lines.append(f"- saturation: {q['saturation']} (mean={self.saturation_mean:.2f}, color_cast={self.color_
    cast:.2f})")
        lines.append(f"- edge density: {q['edge_density']} ({self.edge_density:.3f})")
        lines.append(f"- high-frequency energy: {q['high_frequency_energy']} ({self.high_freq_energy:.3f})")
        lines.append(f"- noise estimate: {q['noise_estimate']} (sigma={self.noise_sigma:.1f})")
        lines.append(f"- blur estimate: {q['blur_estimate']} (laplacian_var={self.blur_score:.1f})")
        lines.append(f"- jpeg blockiness: {q['jpeg_blockiness']} ({self.blockiness:.3f})")
        lines.append(f"- haze estimate: {q['haze_estimate']} (dark_channel={self.haze_score:.2f})")
        return "\n".join(lines)
def _to_rgb_array(image) -> np.ndarray:
    if isinstance(image, Image.Image):
```

```
        return np.array(image.convert("RGB"))
    arr = np.asarray(image)
    if arr.ndim == 2:
        arr = np.stack([arr] * 3, axis=-1)
    return arr[..., :3].astype(np.uint8)
def _estimate_noise_sigma(gray: np.ndarray) -> float:
    """中值滤波残差的 MAD 鲁棒估计噪声标准差。"""
    med = cv2.medianBlur(gray, 3)
    residual = gray.astype(np.float32) - med.astype(np.float32)
    mad = np.median(np.abs(residual - np.median(residual)))
    return float(1.4826 * mad)
def _high_freq_energy(gray: np.ndarray) -> float:
    """高频（半径 > 1/4 频谱宽度）能量占总能量的比例。"""
    f = np.fft.fftshift(np.fft.fft2(gray.astype(np.float32)))
    power = np.abs(f) ** 2
    h, w = gray.shape
    cy, cx = h // 2, w // 2
    yy, xx = np.ogrid[:h, :w]
    radius = np.sqrt((yy - cy) ** 2 + (xx - cx) ** 2)
    mask = radius > min(h, w) / 4.0
    total = float(power.sum()) + 1e-8
    return float(power[mask].sum() / total)
def _blockiness(gray: np.ndarray) -> float:
    """8x8 块边界处与非边界处水平/垂直梯度差异，衡量 JPEG 块效应。"""
    g = gray.astype(np.float32)
    dh = np.abs(np.diff(g, axis=1))
    dv = np.abs(np.diff(g, axis=0))
    if dh.shape[1] < 16 or dv.shape[0] < 16:
        return 0.0
    bh = dh[:, 7::8].mean()
    bv = dv[7::8, :].mean()
    nh = np.delete(dh, np.s_[7::8], axis=1).mean()
    nv = np.delete(dv, np.s_[7::8], axis=0).mean()
    return float(max(0.0, (bh - nh) + (bv - nv)) / 255.0)
def _dark_channel(rgb: np.ndarray, patch: int = 15) -> float:
    """暗通道先验均值，用于雾浓度估计。"""
    mn = rgb.min(axis=2)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (patch, patch))
    dark = cv2.erode(mn, kernel)
    return float(dark.mean() / 255.0)
def compute_image_statistics(image) -> ImageStatistics:
    """计算图像低层统计先验。输入 PIL 图像或 RGB uint8 数组。"""
    rgb = _to_rgb_array(image)
    h, w = rgb.shape[:2]
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)
    brightness = float(gray.mean())
    dark_ratio = float((gray < 40).mean())
    highlight_ratio = float((gray > 220).mean())
    contrast = float(gray.std())
```

```
    saturation = float(hsv[..., 1].mean() / 255.0)
    chan_means = rgb.reshape(-1, 3).mean(axis=0)
    color_cast = float((chan_means.max() - chan_means.min()) / 255.0)
    edges = cv2.Canny(gray, 100, 200)
    edge_density = float((edges > 0).mean())
    blur_score = float(cv2.Laplacian(gray, cv2.CV_64F).var())
    return ImageStatistics(
        width=w,
        height=h,
        brightness_mean=brightness,
        dark_ratio=dark_ratio,
        highlight_ratio=highlight_ratio,
        contrast_global=contrast,
        saturation_mean=saturation,
        color_cast=color_cast,
        edge_density=edge_density,
        high_freq_energy=_high_freq_energy(gray),
        noise_sigma=_estimate_noise_sigma(gray),
        blur_score=blur_score,
        blockiness=_blockiness(gray),
        haze_score=_dark_channel(rgb),
    )
# -*- coding: utf-8 -*-
"""复原工具箱：工具规格注册表、本地经典算子、远程专家模型服务客户端。"""
from .registry import ToolSpec, TOOL_SPECS, get_tool_spec, tools_for_degradation, degradation_of_tool, prompt_vi
    sible_tools
from .toolbox import Toolbox, build_toolbox, ToolExecutionError
from .client import RemoteToolClient, ToolServiceLoadBalancer
__all__ = [
    "ToolSpec", "TOOL_SPECS", "get_tool_spec", "tools_for_degradation", "degradation_of_tool", "prompt_visible_t
    ools",
    "Toolbox", "build_toolbox", "ToolExecutionError",
    "RemoteToolClient", "ToolServiceLoadBalancer",
]
# -*- coding: utf-8 -*-
"""
复原工具规格注册表。
每个工具对应一个预训练专家模型（或经典算子）的一个任务头，
记录其可处理的退化类型、所属模型家族、远程服务端口、参数模式与相对计算代价。
同一退化类型可配置多个候选工具，由智能体依据图像统计先验自适应选择。
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
# 远程专家模型服务的默认端口分配（每个模型家族一个服务进程）
FAMILY_PORTS: Dict[str, int] = {
    "SwinIR": 5001,
    "DehazeFormer": 5002,
    "DRBNet": 5003,
    "MPRNet": 5004,
```

```
    "FBCNN": 5005,
    "Restormer": 5006,
    "XRestormer": 5007,
    "SCUNet": 5008,
    "Retinexformer": 5009,
    "HAT": 5010,
    "NeRD": 5011,
    "NAFNet": 5012,
    "DiffBIR": 5013,
    "Classical": 0,  # 本地经典算子，无需远程服务
}
@dataclass(frozen=True)
class ToolSpec:
    """复原工具规格：名称、目标退化、模型家族、说明、参数模式与相对代价。"""
    name: str
    degradation: str
    family: str
    description: str
    parameters: Dict[str, Dict[str, Any]] = field(default_factory=dict)
    cost: float = 1.0
    prompt_visible: bool = True
    @property
    def port(self) -> int:
        """该工具所属模型家族的服务端口，0 表示本地算子。"""
        return FAMILY_PORTS[self.family]
    @property
    def is_local(self) -> bool:
        """是否为本地算子。"""
        return self.port == 0
    def normalize_arguments(self, arguments: Optional[Dict[str, Any]]) -> Dict[str, Any]:
        """填充默认值并把数值参数裁剪到合法区间；忽略未定义参数。"""
        arguments = arguments or {}
        out: Dict[str, Any] = {}
        for key, schema in self.parameters.items():
            value = arguments.get(key, schema.get("default"))
            if value is None:
                continue
            if schema.get("type") in ("number", "integer"):
                try:
                    value = float(value)
                except (TypeError, ValueError):
                    value = float(schema.get("default", 0))
                if "min" in schema:
                    value = max(float(schema["min"]), value)
                if "max" in schema:
                    value = min(float(schema["max"]), value)
                if "choices" in schema:
                    value = min(schema["choices"], key=lambda c: abs(c - value))
                if schema.get("type") == "integer":
                    value = int(round(value))
```

```
            out[key] = value
        return out
    def prompt_line(self) -> str:
        """提示词中的一行工具说明。"""
        if self.parameters:
            params = ", ".join(
                f'"{k}": {s.get("hint", s.get("type"))}' for k, s in self.parameters.items()
            )
            params = "{ " + params + " }"
        else:
            params = "{}"
        return f'- `{self.name}`: `{params}` — For "{self.degradation}". {self.description}'
_STRENGTH = {"strength": {"type": "number", "min": 0.0, "max": 1.0, "default": 0.5, "hint": "0..1"}}
_JPEG_Q = {"jpeg": {"type": "integer", "min": 1, "max": 100, "default": 30, "hint": "1..100"}}
_SCALE = {"scale": {"type": "integer", "choices": [2, 3, 4], "default": 2, "hint": "2|3|4"}}
_GAMMA = {"gamma": {"type": "number", "min": 0.2, "max": 1.0, "default": 0.5, "hint": "0.2..1.0"}}
_SHIFT = {"shift": {"type": "integer", "min": 0, "max": 120, "default": 40, "hint": "0..120"}}
TOOL_SPECS: Dict[str, ToolSpec] = {s.name: s for s in [
    # ---- 去雾 ----
    ToolSpec("dehazeformer_dehaze", "haze", "DehazeFormer", "Transformer dehazing, robust to dense haze.", _STRE
    NGTH, 1.2),
    # ---- 去噪（多候选）----
    ToolSpec("scunet_real_denoising_psnr", "noise", "SCUNet", "Real-noise denoiser optimised for PSNR; smooth ou
    tput, good for heavy noise.", {}, 1.0),
    ToolSpec("scunet_real_denoising_gan", "noise", "SCUNet", "Real-noise denoiser with GAN loss; preserves textu
    re, good for detailed images.", {}, 1.0),
    ToolSpec("swinir_denoising", "noise", "SwinIR", "Gaussian-noise denoiser; strong on moderate synthetic noise
    .", {}, 1.1),
    ToolSpec("mprnet_denoising", "noise", "MPRNet", "Multi-stage denoiser; balanced detail and smoothness.", {},
     1.1, prompt_visible=False),
    ToolSpec("scunet_color_denoising", "noise", "SCUNet", "Synthetic Gaussian-noise colour denoiser (levels 15/2
    5/50).", {}, 1.0),
    # ---- 运动去模糊（多候选）----
    ToolSpec("restormer_motion_deblurring", "motion blur", "Restormer", "General motion deblurring, strong on la
    rge blur kernels.", {}, 1.3),
    ToolSpec("nafnet_deblur", "motion blur", "NAFNet", "Lightweight motion deblurring; preserves fine edges on s
    harp-textured images.", {}, 0.8),
    ToolSpec("mprnet_motion_deblurring", "motion blur", "MPRNet", "Multi-stage motion deblurring.", {}, 1.1, pro
    mpt_visible=False),
    ToolSpec("xrestormer_motion_deblurring", "motion blur", "XRestormer", "Motion deblurring variant with channe
    l attention.", {}, 1.3, prompt_visible=False),
    # ---- 散焦去模糊（多候选）----
    ToolSpec("restormer_defocus_deblurring", "defocus blur", "Restormer", "Defocus deblurring for smooth, low-ed
    ge-density images.", {}, 1.3),
    ToolSpec("drbnet_defocus_deblurring", "defocus blur", "DRBNet", "Defocus deblurring that recovers fine detai
    l on high-edge-density images.", {}, 1.0),
    # ---- 去雨（多候选）----
    ToolSpec("restormer_deraining", "rain", "Restormer", "Rain-streak removal with strong structure preservation
    .", {}, 1.3),
    ToolSpec("mprnet_deraining", "rain", "MPRNet", "Rain-streak removal; good for dense rain.", {}, 1.1),
```

```
    ToolSpec("xrestormer_deraining", "rain", "XRestormer", "Rain-streak removal with channel attention; robust o
    n heavy rain.", {}, 1.3),
    ToolSpec("nerd_deraining", "rain", "NeRD", "Rain removal with directional priors.", {}, 1.2, prompt_visible=
    False),
    # ---- JPEG 伪影（多候选）----
    ToolSpec("fbcnn_jpeg_artifact_removal", "jpeg compression artifact", "FBCNN", "Flexible blind JPEG artifact
    removal; quality factor controllable.", _JPEG_Q, 0.9),
    ToolSpec("swinir_jpeg_artifact_removal", "jpeg compression artifact", "SwinIR", "JPEG artifact removal with
    detail recovery.", {}, 1.1, prompt_visible=False),
    # ---- 超分辨率（多候选）----
    ToolSpec("swinir_super_resolution", "low resolution", "SwinIR", "Robust real-world super-resolution.", _SCAL
    E, 1.5),
    ToolSpec("xrestormer_super_resolution", "low resolution", "XRestormer", "Super-resolution with channel atten
    tion; keeps fine texture on detail-rich images.", _SCALE, 1.4),
    ToolSpec("diffbir_super_resolution", "low resolution", "DiffBIR", "Diffusion-based blind super-resolution; h
    allucinates rich detail at high cost.", _SCALE, 3.0),
    ToolSpec("hat_super_resolution", "low resolution", "HAT", "Super-resolution with strong texture hallucinatio
    n for detail-rich images.", _SCALE, 1.8, prompt_visible=False),
    # ---- 低光增强（多候选）----
    ToolSpec("retinexformer_sdsd_indoor", "dark", "Retinexformer", "Retinex-based low-light enhancement for indo
    or/low-noise scenes.", {}, 1.0),
    ToolSpec("retinexformer_fivek", "dark", "Retinexformer", "Retinexformer trained on FiveK; natural exposure f
    or photographic scenes.", {}, 1.0),
    ToolSpec("retinexformer_enhance", "dark", "Retinexformer", "General low-light enhancement.", {}, 1.0, prompt
    _visible=False),
    ToolSpec("gamma_correction", "dark", "Classical", "Fast gamma brightening without learned priors.", _GAMMA,
    0.1, prompt_visible=False),
    ToolSpec("histogram_equalization", "dark", "Classical", "CLAHE contrast/brightness enhancement.", {}, 0.1, p
    rompt_visible=False),
    ToolSpec("constant_shift", "dark", "Classical", "Constant brightness shift.", _SHIFT, 0.05, prompt_visible=F
    alse),
]}
def get_tool_spec(name: str) -> ToolSpec:
    """按名称获取工具规格，不存在时抛出 KeyError。"""
    if name not in TOOL_SPECS:
        raise KeyError(f"未注册的工具: {name}")
    return TOOL_SPECS[name]
def tools_for_degradation(degradation: str, visible_only: bool = False) -> List[ToolSpec]:
    """返回处理指定退化类型的候选工具列表。"""
    return [s for s in TOOL_SPECS.values() if s.degradation == degradation and (s.prompt_visible or not visible_
    only)]
def degradation_of_tool(name: str) -> Optional[str]:
    """返回工具对应的退化类型，未知工具返回 None。"""
    spec = TOOL_SPECS.get(name)
    return spec.degradation if spec else None
def prompt_visible_tools() -> List[ToolSpec]:
    """返回写入智能体提示词的工具列表。"""
    return [s for s in TOOL_SPECS.values() if s.prompt_visible]
def all_degradations() -> List[str]:
    """返回注册表覆盖的全部退化类型。"""
```

```
    seen: List[str] = []
    for s in TOOL_SPECS.values():
        if s.degradation not in seen:
            seen.append(s.degradation)
    return seen
# -*- coding: utf-8 -*-
"""
本地经典复原算子。
两个用途：
（1）BrighteningToolbox 等本身即为经典算子的工具；
（2）在无 GPU / 无远程专家模型服务时，为每个注册工具提供可运行的经典近似实现，
    保证训练环境、推理流程与单元测试在离线条件下完整可运行。
同一退化类型下不同候选工具采用不同参数或算法组合，保留「候选工具互补」的语义。
"""
from __future__ import annotations
from typing import Any, Callable, Dict
import cv2
import numpy as np
Op = Callable[[np.ndarray, Dict[str, Any]], np.ndarray]
def _unsharp(img: np.ndarray, sigma: float, amount: float) -> np.ndarray:
    blur = cv2.GaussianBlur(img, (0, 0), sigma)
    return cv2.addWeighted(img, 1.0 + amount, blur, -amount, 0)
def _clip(img: np.ndarray) -> np.ndarray:
    return np.clip(img, 0, 255).astype(np.uint8)
# ---------- 去噪 ----------
def denoise_nlmeans(img, params, h=10.0, post_sharpen=0.0):
    """非局部均值去噪，h 控制强度，可选后置锐化。"""
    out = cv2.fastNlMeansDenoisingColored(img, None, h, h, 7, 21)
    return _unsharp(out, 1.0, post_sharpen) if post_sharpen > 0 else out
def denoise_bilateral(img, params):
    """双边滤波去噪。"""
    return cv2.bilateralFilter(img, 7, 50, 50)
# ---------- 去模糊 ----------
def deblur_wiener_like(img, params, radius=1.5, amount=1.2):
    """基于反锐化掩模的去模糊近似。"""
    return _clip(_unsharp(img, radius, amount))
def deblur_richardson_lucy(img, params, iterations=8, ksize=5):
    """简化的 Richardson-Lucy 反卷积（均值核）。"""
    psf = np.ones((ksize, ksize), dtype=np.float32) / (ksize * ksize)
    x = img.astype(np.float32) / 255.0
    est = x.copy()
    for _ in range(iterations):
        conv = cv2.filter2D(est, -1, psf)
        ratio = x / (conv + 1e-6)
        est = est * cv2.filter2D(ratio, -1, psf)
    return _clip(est * 255.0)
# ---------- 去雾 ----------
def dehaze_dark_channel(img, params):
    """暗通道先验去雾，strength 控制去雾强度 omega。"""
    strength = float(params.get("strength", 0.5))
```

```
    omega = 0.6 + 0.35 * strength
    x = img.astype(np.float32) / 255.0
    mn = x.min(axis=2)
    dark = cv2.erode(mn, cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15)))
    flat = dark.reshape(-1)
    idx = np.argsort(flat)[-max(1, int(0.001 * flat.size)):]
    airlight = x.reshape(-1, 3)[idx].mean(axis=0)
    t = 1.0 - omega * cv2.erode((x / (airlight + 1e-6)).min(axis=2), cv2.getStructuringElement(cv2.MORPH_RECT, (
    15, 15)))
    t = cv2.GaussianBlur(t, (0, 0), 8)
    t = np.clip(t, 0.1, 1.0)[..., None]
    out = (x - airlight) / t + airlight
    return _clip(out * 255.0)
# ---------- 去雨 ----------
def derain_median(img, params, ksize=5):
    """中值滤波去除细雨线。"""
    return cv2.medianBlur(img, ksize)
def derain_guided(img, params):
    """方向中值 + 双边滤波组合去除细雨线。"""
    out = cv2.medianBlur(img, 3)
    return cv2.bilateralFilter(out, 9, 40, 40)
# ---------- 超分辨率 ----------
def sr_bicubic_sharpen(img, params, amount=0.6):
    """双三次放大并锐化的超分辨率近似。"""
    scale = int(params.get("scale", 2))
    h, w = img.shape[:2]
    up = cv2.resize(img, (w * scale, h * scale), interpolation=cv2.INTER_CUBIC)
    return _clip(_unsharp(up, 1.0, amount))
def sr_lanczos(img, params):
    """Lanczos 插值放大。"""
    scale = int(params.get("scale", 2))
    h, w = img.shape[:2]
    return cv2.resize(img, (w * scale, h * scale), interpolation=cv2.INTER_LANCZOS4)
# ---------- 低光增强 ----------
def enhance_gamma(img, params):
    """gamma 校正增亮。"""
    gamma = float(params.get("gamma", 0.5))
    lut = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)], dtype=np.uint8)
    return cv2.LUT(img, lut)
def enhance_clahe(img, params):
    """在 LAB 亮度通道上做限制对比度自适应直方图均衡。"""
    lab = cv2.cvtColor(img, cv2.COLOR_RGB2LAB)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    lab[..., 0] = clahe.apply(lab[..., 0])
    return cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)
def enhance_constant_shift(img, params):
    """常数平移增亮。"""
    shift = int(params.get("shift", 40))
    return _clip(img.astype(np.int16) + shift)
def enhance_retinex(img, params, sigma=30.0):
```

```
    """单尺度 Retinex：log(I) - log(blur(I))，再归一化。"""
    x = img.astype(np.float32) + 1.0
    illum = cv2.GaussianBlur(x, (0, 0), sigma)
    r = np.log(x) - np.log(illum + 1.0)
    r = (r - r.min()) / (r.max() - r.min() + 1e-6)
    return _clip(r * 255.0)
# ---------- JPEG 伪影 ----------
def deblock_bilateral(img, params):
    """双边滤波去块效应。"""
    return cv2.bilateralFilter(img, 5, 30, 30)
def deblock_nlmeans(img, params):
    """非局部均值去块效应。"""
    return cv2.fastNlMeansDenoisingColored(img, None, 5, 5, 7, 21)
# 工具名 → 经典近似实现；同类工具使用不同算法/参数以保留互补性
CLASSICAL_OPS: Dict[str, Op] = {
    "dehazeformer_dehaze": dehaze_dark_channel,
    "scunet_real_denoising_psnr": lambda img, p: denoise_nlmeans(img, p, h=12.0),
    "scunet_real_denoising_gan": lambda img, p: denoise_nlmeans(img, p, h=8.0, post_sharpen=0.3),
    "swinir_denoising": lambda img, p: denoise_nlmeans(img, p, h=10.0),
    "mprnet_denoising": denoise_bilateral,
    "restormer_motion_deblurring": lambda img, p: deblur_richardson_lucy(img, p, iterations=10, ksize=7),
    "nafnet_deblur": lambda img, p: deblur_wiener_like(img, p, radius=1.2, amount=1.0),
    "mprnet_motion_deblurring": lambda img, p: deblur_richardson_lucy(img, p, iterations=6, ksize=5),
    "xrestormer_motion_deblurring": lambda img, p: deblur_wiener_like(img, p, radius=2.0, amount=1.4),
    "restormer_defocus_deblurring": lambda img, p: deblur_richardson_lucy(img, p, iterations=8, ksize=5),
    "drbnet_defocus_deblurring": lambda img, p: deblur_wiener_like(img, p, radius=1.5, amount=1.5),
    "restormer_deraining": derain_guided,
    "mprnet_deraining": lambda img, p: derain_median(img, p, ksize=5),
    "xrestormer_deraining": lambda img, p: derain_median(img, p, ksize=3),
    "nerd_deraining": derain_guided,
    "fbcnn_jpeg_artifact_removal": deblock_bilateral,
    "swinir_jpeg_artifact_removal": deblock_nlmeans,
    "swinir_super_resolution": sr_bicubic_sharpen,
    "xrestormer_super_resolution": lambda img, p: sr_bicubic_sharpen(img, p, amount=0.8),
    "diffbir_super_resolution": sr_lanczos,
    "scunet_color_denoising": lambda img, p: denoise_nlmeans(img, p, h=9.0),
    "retinexformer_fivek": lambda img, p: enhance_retinex(img, p, sigma=40.0),
    "hat_super_resolution": lambda img, p: sr_bicubic_sharpen(img, p, amount=1.0),
    "retinexformer_sdsd_indoor": lambda img, p: enhance_retinex(img, p, sigma=30.0),
    "retinexformer_enhance": lambda img, p: enhance_retinex(img, p, sigma=50.0),
    "gamma_correction": enhance_gamma,
    "histogram_equalization": enhance_clahe,
    "constant_shift": enhance_constant_shift,
}
def run_classical(tool_name: str, img: np.ndarray, params: Dict[str, Any]) -> np.ndarray:
    """按工具名运行对应的经典近似算子。"""
    if tool_name not in CLASSICAL_OPS:
        raise KeyError(f"工具 {tool_name} 没有经典近似实现")
    return CLASSICAL_OPS[tool_name](img, params)
# -*- coding: utf-8 -*-
```

```
"""
远程复原工具服务客户端。
协议（与工具服务端 restoramind.tool_server 一致）：
  POST http://{ip}:{port}/process
    multipart: image=<PNG 字节>
    form:      task=<工具名>, 其余为工具参数
  响应 JSON: {"success": true, "image": "<base64 PNG>"} 或 {"success": false, "error": "..."}
支持多服务器轮询/随机负载均衡（环境变量 TOOL_SERVICE_IPS 配置地址，逗号分隔；TOOL_LOAD_BALANCE_STRATEGY 选择策略）。
"""
from __future__ import annotations
import base64
import io
import os
import random
import threading
from typing import Any, Dict, List, Optional
import requests
from PIL import Image
class ToolServiceLoadBalancer:
  """多 IP 负载均衡器。"""
  def __init__(self, ips: Optional[List[str]] = None, strategy: Optional[str] = None):
    if ips is None:
      raw = os.environ.get("TOOL_SERVICE_IPS", "").strip()
      if raw:
        ips = [x.strip() for x in raw.replace(";", ",").split(",") if x.strip()]
      else:
        ips = [os.environ.get("TOOL_SERVICE_IP", "127.0.0.1").strip()]
    if not ips:
      raise ValueError("至少需要一个工具服务 IP")
    self._ips = list(ips)
    # 策略优先级：构造参数 > 环境变量 TOOL_LOAD_BALANCE_STRATEGY > round_robin
    strategy = strategy or os.environ.get("TOOL_LOAD_BALANCE_STRATEGY", "round_robin").strip().lower()
    self._strategy = strategy if strategy in ("round_robin", "random") else "round_robin"
    self._index = 0
    self._lock = threading.Lock()
  @property
  def ips(self) -> List[str]:
    """已配置的服务器地址列表。"""
    return list(self._ips)
  def next_ip(self) -> str:
    """按策略返回下一个服务器地址。"""
    if len(self._ips) == 1:
      return self._ips[0]
    if self._strategy == "random":
      return random.choice(self._ips)
    with self._lock:
      ip = self._ips[self._index % len(self._ips)]
      self._index += 1
    return ip
```

```
def encode_image(image: Image.Image) -> bytes:
    """把图像编码为 PNG 字节。"""
    buf = io.BytesIO()
    image.convert("RGB").save(buf, format="PNG")
    return buf.getvalue()
def decode_image_b64(data: str) -> Image.Image:
    """解码 base64 编码的 PNG 图像。"""
    return Image.open(io.BytesIO(base64.b64decode(data))).convert("RGB")
class RemoteToolClient:
    """通过 HTTP 调用远程专家模型服务。"""
    def __init__(self, balancer: Optional[ToolServiceLoadBalancer] = None, timeout: float = 300.0,
                 session: Optional[Any] = None):
        self.balancer = balancer or ToolServiceLoadBalancer()
        self.timeout = timeout
        # session 可注入（例如 Flask test_client 适配器）以便离线测试
        self.session = session or requests.Session()
    def url_for(self, port: int) -> str:
        """构造指定端口的服务请求地址（含负载均衡）。"""
        return f"http://{self.balancer.next_ip()}:{port}/process"
    def process(self, image: Image.Image, port: int, task: str, params: Dict[str, Any]) -> Image.Image:
        """向远程服务提交图像并返回复原结果。"""
        files = {"image": ("image.png", encode_image(image), "image/png")}
        data = {"task": task, **{k: str(v) for k, v in params.items()}}
        resp = self.session.post(self.url_for(port), files=files, data=data, timeout=self.timeout)
        resp.raise_for_status()
        result = resp.json()
        if not result.get("success"):
            raise RuntimeError(f"工具服务返回错误: {result.get('error', 'unknown')}")
        return decode_image_b64(result["image"])
# -*- coding: utf-8 -*-
"""
工具箱：智能体的离散动作空间。
Toolbox 统一对外提供 ``execute(tool_name, image, arguments) -> image`` ，
内部根据工具规格决定走本地经典算子还是远程专家模型服务。
"""
from __future__ import annotations
import time
from dataclasses import dataclass
from typing import Any, Callable, Dict, List, Optional
import numpy as np
from PIL import Image
from .classical import run_classical
from .client import RemoteToolClient
from .registry import TOOL_SPECS, ToolSpec, get_tool_spec
Executor = Callable[[ToolSpec, Image.Image, Dict[str, Any]], Image.Image]
class ToolExecutionError(RuntimeError):
    """工具执行失败。"""
    pass
@dataclass
class ToolResult:
```

```
    """一次工具执行的结果：图像、工具名、实际参数与耗时。"""
    image: Image.Image
    tool_name: str
    arguments: Dict[str, Any]
    elapsed: float
class Toolbox:
    """智能体的离散动作空间，统一对外提供工具执行。"""
    def __init__(self, executor: Executor, specs: Optional[Dict[str, ToolSpec]] = None):
        self._executor = executor
        self._specs = specs or TOOL_SPECS
        self.call_log: List[ToolResult] = []
    @property
    def tool_names(self) -> List[str]:
        """工具箱中的全部工具名。"""
        return list(self._specs.keys())
    def has(self, name: str) -> bool:
        """工具箱是否包含指定工具。"""
        return name in self._specs
    def spec(self, name: str) -> ToolSpec:
        """获取工具规格，不存在时抛出 ToolExecutionError。"""
        if name not in self._specs:
            raise ToolExecutionError(f"工具箱中不存在工具: {name}")
        return self._specs[name]
    def execute(self, tool_name: str, image: Image.Image, arguments: Optional[Dict[str, Any]] = None) -> ToolRes
    ult:
        """规范化参数并执行工具，返回 ToolResult。"""
        spec = self.spec(tool_name)
        params = spec.normalize_arguments(arguments)
        start = time.time()
        try:
            out = self._executor(spec, image, params)
        except Exception as exc:  # noqa: BLE001 - 统一包装为工具执行错误
            raise ToolExecutionError(f"{tool_name} 执行失败: {exc}") from exc
        result = ToolResult(out.convert("RGB"), tool_name, params, time.time() - start)
        self.call_log.append(result)
        return result
def _classical_executor(spec: ToolSpec, image: Image.Image, params: Dict[str, Any]) -> Image.Image:
    arr = np.array(image.convert("RGB"))
    return Image.fromarray(run_classical(spec.name, arr, params))
def _remote_executor_factory(client: RemoteToolClient) -> Executor:
    def _exec(spec: ToolSpec, image: Image.Image, params: Dict[str, Any]) -> Image.Image:
        if spec.is_local:
            return _classical_executor(spec, image, params)
        return client.process(image, spec.port, spec.name, params)
    return _exec
def build_toolbox(mode: str = "local", client: Optional[RemoteToolClient] = None,
                  specs: Optional[Dict[str, ToolSpec]] = None) -> Toolbox:
    """
    mode = "local" ：全部工具使用本地经典算子（离线 / 测试）
    mode = "remote"：深度专家模型走远程服务，经典算子本地执行（训练 / 部署）
```

```
    """
    if mode == "local":
        return Toolbox(_classical_executor, specs)
    if mode == "remote":
        return Toolbox(_remote_executor_factory(client or RemoteToolClient()), specs)
    raise ValueError(f"未知工具箱模式: {mode}")
# -*- coding: utf-8 -*-
"""复原工具服务端。"""
from .backends import Backend, ClassicalBackend, CallableBackend, load_backend
from .app import create_app
__all__ = ["Backend", "ClassicalBackend", "CallableBackend", "load_backend", "create_app"]
# -*- coding: utf-8 -*-
"""
工具服务端后端。
Backend 把「工具名 + 图像 + 参数」映射为复原图像。
    ClassicalBackend : 经典算子实现，开箱即用
    CallableBackend  : 以 ``module:function`` 形式加载任意用户实现（例如封装
                       SwinIR / Restormer / SCUNet 等预训练模型的推理函数），
                       函数签名须为 fn(tool_name: str, image: PIL.Image, params: dict) -> PIL.Image
"""
from __future__ import annotations
import importlib
from abc import ABC, abstractmethod
from typing import Any, Callable, Dict
import numpy as np
from PIL import Image
from ..tools.classical import run_classical
class Backend(ABC):
    """工具服务后端抽象：把（工具名，图像，参数）映射为复原图像。"""
    @abstractmethod
    def process(self, tool_name: str, image: Image.Image, params: Dict[str, Any]) -> Image.Image:
        """执行指定工具并返回复原图像。"""
        ...
    def supports(self, tool_name: str) -> bool:  # noqa: D401
        """后端是否支持该工具。"""
        return True
class ClassicalBackend(Backend):
    """经典算子后端，开箱即用。"""
    def process(self, tool_name: str, image: Image.Image, params: Dict[str, Any]) -> Image.Image:
        """使用本地经典算子执行工具。"""
        return Image.fromarray(run_classical(tool_name, np.array(image.convert("RGB")), params))
class CallableBackend(Backend):
    """把任意可调用对象包装为后端，用于接入预训练模型。"""
    def __init__(self, fn: Callable[[str, Image.Image, Dict[str, Any]], Image.Image]):
        self.fn = fn
    def process(self, tool_name: str, image: Image.Image, params: Dict[str, Any]) -> Image.Image:
        """调用用户函数执行工具。"""
        return self.fn(tool_name, image, params).convert("RGB")
def load_backend(spec: str) -> Backend:
    """``classical`` 或 ``package.module:function``。"""
```

```
    if spec == "classical":
        return ClassicalBackend()
    if ":" not in spec:
        raise ValueError("自定义后端需使用 module:function 形式")
    module_name, fn_name = spec.split(":", 1)
    module = importlib.import_module(module_name)
    return CallableBackend(getattr(module, fn_name))
# -*- coding: utf-8 -*-
"""
复原工具 HTTP 服务（Flask）。
每个专家模型家族启动一个服务进程，对外暴露：
    GET  /health          健康检查，返回支持的工具列表
    POST /process         multipart image + form(task, 参数...) → {"success", "image"(base64 PNG)}
"""
from __future__ import annotations
import base64
import io
from typing import Dict, Iterable, Optional
from flask import Flask, jsonify, request
from PIL import Image
from ..tools.registry import TOOL_SPECS, get_tool_spec
from .backends import Backend, ClassicalBackend
def create_app(backend: Optional[Backend] = None, family: Optional[str] = None,
               tools: Optional[Iterable[str]] = None) -> Flask:
    """
    backend : 处理后端，默认经典算子
    family  : 仅服务该模型家族的工具（与端口一一对应）
    tools   : 显式指定服务的工具名集合（优先于 family）
    """
    backend = backend or ClassicalBackend()
    if tools is not None:
        served = {n for n in tools}
    elif family is not None:
        served = {n for n, s in TOOL_SPECS.items() if s.family == family}
    else:
        served = set(TOOL_SPECS.keys())
    app = Flask("restoramind_tool_server")
    app.config["SERVED_TOOLS"] = sorted(served)
    @app.get("/health")
    def health():
        """健康检查，返回服务支持的工具列表。"""
        return jsonify({"status": "ok", "tools": app.config["SERVED_TOOLS"]})
    @app.post("/process")
    def process():
        """处理一次复原请求并返回 base64 编码的结果图。"""
        task = request.form.get("task", "")
        if task not in served:
            return jsonify({"success": False, "error": f"unsupported task '{task}'"}), 400
        if "image" not in request.files:
            return jsonify({"success": False, "error": "missing image"}), 400
```

```
        try:
            image = Image.open(io.BytesIO(request.files["image"].read())).convert("RGB")
            spec = get_tool_spec(task)
            params: Dict[str, str] = {k: v for k, v in request.form.items() if k != "task"}
            out = backend.process(task, image, spec.normalize_arguments(params))
            buf = io.BytesIO()
            out.save(buf, format="PNG")
            return jsonify({"success": True, "image": base64.b64encode(buf.getvalue()).decode("ascii"),
                            "width": out.width, "height": out.height})
        except Exception as exc:  # noqa: BLE001
            return jsonify({"success": False, "error": str(exc)}), 500
    return app
# -*- coding: utf-8 -*-
"""智能体：响应解析、提示词、复原环境、策略与轨迹采样。"""
from .parser import ToolCall, ParsedResponse, parse_response, split_turns, format_tool_call, format_answer
from .prompts import build_system_prompt, build_first_user_prompt, build_feedback_prompt, MODES
from .env import EnvConfig, RestorationEnv, Observation, AgentState
from .trajectory import Trajectory, StepRecord
from .policy import Policy, HeuristicPolicy, OpenAICompatiblePolicy
from .runner import run_episode
__all__ = [
    "ToolCall", "ParsedResponse", "parse_response", "split_turns", "format_tool_call", "format_answer",
    "build_system_prompt", "build_first_user_prompt", "build_feedback_prompt", "MODES",
    "EnvConfig", "RestorationEnv", "Observation", "AgentState",
    "Trajectory", "StepRecord",
    "Policy", "HeuristicPolicy", "OpenAICompatiblePolicy",
    "run_episode",
]
# -*- coding: utf-8 -*-
"""
智能体响应解析。
响应格式（严格）：
    <think>推理过程</think>
    <tool_call>[{"name": 工具名, "degradation": 退化类型, "arguments": {...}, "rollback": false}]</tool_call>
或终止：
    <think>...</think>
    <answer>{"restoration_log": ["rain", "dark", ...]}</answer>
"""
from __future__ import annotations
import json
import re
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
THINK_RE = re.compile(r"<think>(.*?)</think>", re.DOTALL)
TOOL_CALL_RE = re.compile(r"<tool_call>\s*(.*?)\s*</tool_call>", re.DOTALL)
ANSWER_RE = re.compile(r"<answer>\s*(.*?)\s*</answer>", re.DOTALL)
@dataclass
class ToolCall:
    """单个工具调用：工具名、目标退化类型、参数、是否回退重试。"""
    name: str
```

```
    degradation: Optional[str] = None
    arguments: Dict[str, Any] = field(default_factory=dict)
    rollback: bool = False
    def to_dict(self) -> dict:
        """转换为可序列化的字典。"""
        d = {"name": self.name, "degradation": self.degradation, "arguments": self.arguments}
        if self.rollback:
            d["rollback"] = True
        return d
@dataclass
class ParsedResponse:
    """单轮响应的解析结果，含推理文本、工具调用、最终答案与解析错误。"""
    think: Optional[str]
    tool_calls: List[ToolCall]
    answer: Optional[Dict[str, Any]]
    errors: List[str] = field(default_factory=list)
    raw: str = ""
    @property
    def is_answer(self) -> bool:
        """该轮是否为最终答案。"""
        return self.answer is not None
    @property
    def is_valid(self) -> bool:
        """该轮是否格式有效（无错误且含工具调用或答案）。"""
        return not self.errors and (self.is_answer or bool(self.tool_calls))
def _parse_tool_calls(payload: str) -> List[ToolCall]:
    data = json.loads(payload)
    if isinstance(data, dict):
        data = [data]
    if not isinstance(data, list) or not data:
        raise ValueError("tool_call 必须是非空 JSON 列表")
    calls: List[ToolCall] = []
    for item in data:
        if not isinstance(item, dict) or "name" not in item:
            raise ValueError("tool_call 元素缺少 name 字段")
        args = item.get("arguments", {})
        if not isinstance(args, dict):
            raise ValueError("arguments 必须是对象")
        calls.append(ToolCall(
            name=str(item["name"]),
            degradation=item.get("degradation"),
            arguments=args,
            rollback=bool(item.get("rollback", False)),
        ))
    return calls
def parse_response(text: str) -> ParsedResponse:
    """解析单轮响应。解析失败时在 errors 中给出原因，不抛异常。"""
    errors: List[str] = []
    think_m = THINK_RE.search(text)
    think = think_m.group(1).strip() if think_m else None
```

```
    if think is None:
        errors.append("missing <think>")
    tool_m = TOOL_CALL_RE.search(text)
    ans_m = ANSWER_RE.search(text)
    if tool_m and ans_m:
        errors.append("both <tool_call> and <answer> present")
    tool_calls: List[ToolCall] = []
    answer: Optional[Dict[str, Any]] = None
    if tool_m:
        try:
            tool_calls = _parse_tool_calls(tool_m.group(1))
        except (json.JSONDecodeError, ValueError) as exc:
            errors.append(f"invalid tool_call: {exc}")
    if ans_m:
        try:
            answer = json.loads(ans_m.group(1))
            if not isinstance(answer, dict) or "restoration_log" not in answer:
                errors.append("answer must be an object with restoration_log")
                answer = None
        except json.JSONDecodeError as exc:
            errors.append(f"invalid answer json: {exc}")
    if not tool_m and not ans_m:
        errors.append("neither <tool_call> nor <answer> present")
    return ParsedResponse(think=think, tool_calls=tool_calls, answer=answer, errors=errors, raw=text)
def split_turns(multi_turn_text: str) -> List[str]:
    """把多轮拼接的模型输出按 <think> 起始位置切分为若干轮。"""
    matches = list(THINK_RE.finditer(multi_turn_text))
    turns: List[str] = []
    for i, m in enumerate(matches):
        end = matches[i + 1].start() if i + 1 < len(matches) else len(multi_turn_text)
        turns.append(multi_turn_text[m.start():end])
    return turns
def format_tool_call(think: str, calls: List[ToolCall]) -> str:
    """按严格格式生成包含推理与工具调用的响应文本。"""
    body = json.dumps([c.to_dict() for c in calls], ensure_ascii=False)
    return f"<think>{think}</think>\n<tool_call>{body}</tool_call>"
def format_answer(think: str, restoration_log: List[str]) -> str:
    """按严格格式生成包含推理与 restoration_log 的最终答案文本。"""
    body = json.dumps({"restoration_log": restoration_log}, ensure_ascii=False)
    return f"<think>{think}</think>\n<answer>{body}</answer>"
# -*- coding: utf-8 -*-
"""
提示词模板。
两种对话模式：
    single_tool_iterative：每轮恰好一个工具，逐步决策（序列决策式复原，默认）
    multi_tool_planning  ：每轮给出完整工具序列，系统链式执行后反馈结果
每个决策状态都注入当前图像的低层统计先验，用于同类工具间的自适应选择。
"""
from __future__ import annotations
from typing import Iterable, List, Optional
```

```
from ..tools.registry import ToolSpec, prompt_visible_tools
MODES = ("single_tool_iterative", "multi_tool_planning")
_FORMAT_COMMON = """## Response Format (STRICT)
Every turn must start with a `<think>` block (at least 10 characters) explaining:
- which degradations you believe remain in the current image, and why (use both the image and its statistics);
- which tool you choose among the candidates for that degradation and why the image statistics support it.
Then output exactly ONE of:
1. `<tool_call>[ {{"name": "<tool>", "degradation": "<type>", "arguments": {{...}}}} ]</tool_call>`
2. `<answer>{{"restoration_log": ["<type1>", "<type2>", ...]}}</answer>` (only when restoration is complete; li
   st the degradation types in the order they were handled)
Never output both in one turn. Use only tool names from the list above.
"""
_STATS_NOTE = """## Image Statistics
Each image is accompanied by lightweight low-level statistics (brightness, contrast, edge density,
high-frequency energy, noise / blur / blockiness / haze estimates). Use them to (a) confirm which
degradations remain and (b) choose between tools that handle the same degradation, e.g. prefer
detail-preserving tools when edge density is high and robust/smoothing tools when it is low.
"""
_SINGLE_RULES = """## Rules
- One tool per turn. The tool is applied to the CURRENT image; you will then see the result.
- If the previous tool failed to remove its target degradation, you may retry once with an
  alternative tool of the same type on the PREVIOUS image by adding `"rollback": true`.
- Stop with `<answer>` as soon as no significant degradation remains. Avoid redundant calls.
"""
_MULTI_RULES = """## Rules
- Propose the COMPLETE ordered tool sequence as a JSON list in one `<tool_call>`.
- The system executes the whole sequence from the ORIGINAL image and returns the result.
- If the result is unsatisfactory, propose a different complete sequence; otherwise answer.
"""
def _tool_catalog(specs: Iterable[ToolSpec]) -> str:
    by_deg: dict = {}
    for s in specs:
        by_deg.setdefault(s.degradation, []).append(s)
    lines: List[str] = ["## Available Tools (grouped by degradation type)"]
    for deg, items in by_deg.items():
        lines.append(f"### {deg}")
        lines.extend(s.prompt_line() for s in items)
    return "\n".join(lines)
def build_system_prompt(mode: str = "single_tool_iterative", specs: Optional[Iterable[ToolSpec]] = None) -> str:
    """生成系统提示词：工具目录、统计先验说明、模式规则与严格响应格式。"""
    if mode not in MODES:
        raise ValueError(f"未知对话模式: {mode}")
    specs = list(specs) if specs is not None else prompt_visible_tools()
    intro = ("You are RestoraMind, an image restoration agent. You restore images with unknown, "
             "compound degradations by reasoning about the remaining degradations step by step and "
             "selecting the most suitable expert restoration tool at each step.")
    rules = _SINGLE_RULES if mode == "single_tool_iterative" else _MULTI_RULES
    return "\n\n".join([intro, _tool_catalog(specs), _STATS_NOTE, rules, _FORMAT_COMMON])
def build_first_user_prompt(degradation_hint: Optional[List[str]], stats_text: Optional[str],
                            mode: str = "single_tool_iterative") -> str:
```

```
    """生成首轮用户提示，可包含退化提示与统计先验文本。"""
    parts = ["<image>"]
    if degradation_hint is not None:
        hint = ", ".join(degradation_hint) if degradation_hint else "none"
        parts.append(f"Image analysis indicates these degradation types (order unknown): {hint}")
    else:
        parts.append("Identify the degradations present in this image yourself.")
    if stats_text:
        parts.append(stats_text)
    if mode == "single_tool_iterative":
        parts.append("Select the FIRST tool to apply, or answer if the image is already clean.")
    else:
        parts.append("Propose the complete restoration sequence now.")
    return "\n".join(parts)
def build_feedback_prompt(mode: str, applied: List[str], processed: List[str], remaining: Optional[List[str]],
                          stats_text: Optional[str], error: Optional[str] = None) -> str:
    """生成工具执行后的反馈提示，包含已处理与剩余退化及新图像统计。"""
    parts: List[str] = []
    if error:
        parts.append(f"Tool execution error: {error}. The image is unchanged.")
    else:
        parts.append("Tool(s) applied successfully: " + ", ".join(applied) + ". Here is the processed image:")
    parts.append("<image>")
    if processed:
        parts.append("Processed degradations so far: " + ", ".join(processed))
    if remaining is not None:
        parts.append("Remaining degradations (hint): " + (", ".join(remaining) if remaining else "none"))
    if stats_text:
        parts.append(stats_text)
    if mode == "single_tool_iterative":
        parts.append("Analyse the new image. Continue with the next tool, retry with rollback, or answer if comp
    lete.")
    else:
        parts.append("If satisfactory, answer. Otherwise propose a NEW complete sequence (executed from the orig
    inal image).")
    return "\n".join(parts)
# -*- coding: utf-8 -*-
"""轨迹记录：一次完整复原过程中的状态、动作、工具结果序列。"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
from PIL import Image
from ..data.dataset import Sample
from .parser import ToolCall
@dataclass
class StepRecord:
    """单个决策步：模型响应、解析出的工具调用、执行前后图像索引。"""
    turn: int
    response: str
    tool_calls: List[ToolCall]
```

```
    predicted_degradations: List[str]
    executed_tools: List[str]
    image_index_before: int
    image_index_after: int
    tool_cost: float = 0.0
    error: Optional[str] = None
    format_errors: List[str] = field(default_factory=list)
    is_answer: bool = False
    meta: Dict[str, Any] = field(default_factory=dict)  # 策略可写入 log_prob / 状态键等训练信息
@dataclass
class Trajectory:
    """一次完整复原过程的轨迹：步骤记录、图像历史与最终答案。"""
    sample: Sample
    mode: str
    steps: List[StepRecord] = field(default_factory=list)
    image_history: List[Image.Image] = field(default_factory=list)
    answer: Optional[Dict[str, Any]] = None
    finished: bool = False
    truncated: bool = False
    @property
    def responses(self) -> List[str]:
        """各轮模型响应文本列表。"""
        return [s.response for s in self.steps]
    @property
    def concatenated_response(self) -> str:
        """多轮响应拼接后的文本，用于格式与奖励检查。"""
        return "\n".join(self.responses)
    @property
    def executed_tools(self) -> List[str]:
        """按执行顺序排列的已执行工具名。"""
        return [t for s in self.steps for t in s.executed_tools]
    @property
    def num_tool_calls(self) -> int:
        """模型发出的工具调用总数。"""
        return sum(len(s.tool_calls) for s in self.steps)
    @property
    def final_image(self) -> Image.Image:
        """最终图像（图像历史的最后一张）。"""
        return self.image_history[-1]
    @property
    def restoration_log(self) -> List[str]:
        """最终答案中的复原日志，无答案时为空列表。"""
        if self.answer and isinstance(self.answer.get("restoration_log"), list):
            return [str(x) for x in self.answer["restoration_log"]]
        return []
# -*- coding: utf-8 -*-
"""
图像复原序列决策环境。
状态 S_t = (当前图像 I_t, 统计先验 F_t, 工具调用历史, 退化预测历史, 任务提示)。
动作 a_t = 工具选择（含参数、目标退化、是否回退）。
```

```
转移：执行工具得到 I_{t+1}，重新提取统计先验，更新历史，构造下一状态。
终止：模型输出 <answer>，或达到最大轮数。
环境不计算奖励（由 reward 模块基于轨迹计算），保持单一职责。
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional
from PIL import Image
from ..data.dataset import Sample
from ..priors.image_statistics import ImageStatistics, compute_image_statistics
from ..tools.registry import degradation_of_tool
from ..tools.toolbox import Toolbox, ToolExecutionError
from .parser import ParsedResponse, ToolCall, parse_response
from .prompts import MODES, build_feedback_prompt, build_first_user_prompt, build_system_prompt
from .trajectory import StepRecord, Trajectory
@dataclass
class EnvConfig:
    """环境配置：对话模式、最大轮数、回退次数上限、是否注入统计先验与退化提示。"""
    mode: str = "single_tool_iterative"
    max_turns: int = 6
    max_retries: int = 1            # 同类工具回退重试次数上限（有限回退，而非全局后搜索）
    use_statistics_prior: bool = True
    provide_degradation_hint: bool = True
    max_tools_per_turn: int = 1     # 单工具模式强制为 1；多工具模式 0 表示不限
    stop_on_format_error: bool = False
    def __post_init__(self):
        if self.mode not in MODES:
            raise ValueError(f"未知对话模式: {self.mode}")
        if self.mode == "single_tool_iterative":
            self.max_tools_per_turn = 1
@dataclass
class AgentState:
    """供策略使用的结构化状态（与自然语言提示词内容一致）。"""
    turn: int
    current_image: Image.Image
    statistics: Optional[ImageStatistics]
    degradation_hint: Optional[List[str]]
    processed: List[str]
    remaining: Optional[List[str]]
    tool_history: List[str]
    retries_used: int
    last_error: Optional[str] = None
@dataclass
class Observation:
    """环境观测：多模态消息列表与结构化状态。"""
    messages: List[Dict[str, Any]]
    state: AgentState
def _text_image_content(text: str, image: Image.Image) -> List[Dict[str, Any]]:
    """把含 <image> 占位的文本转为多模态内容片段。"""
    before, _, after = text.partition("<image>")
```

```
    content: List[Dict[str, Any]] = []
    if before.strip():
        content.append({"type": "text", "text": before.strip()})
    content.append({"type": "image", "image": image})
    if after.strip():
        content.append({"type": "text", "text": after.strip()})
    return content
class RestorationEnv:
    """图像复原序列决策环境，负责状态维护、工具执行与状态转移。"""
    def __init__(self, toolbox: Toolbox, config: Optional[EnvConfig] = None):
        self.toolbox = toolbox
        self.config = config or EnvConfig()
        self.system_prompt = build_system_prompt(self.config.mode)
        self._traj: Optional[Trajectory] = None
        self._messages: List[Dict[str, Any]] = []
        self._processed: List[str] = []
        self._retries = 0
        self._turn = 0
        self._last_error: Optional[str] = None
    # ---------------------------------------------------------------------- 状态
    @property
    def trajectory(self) -> Trajectory:
        """当前回合的轨迹记录。"""
        assert self._traj is not None, "请先调用 reset()"
        return self._traj
    @property
    def current_image(self) -> Image.Image:
        """当前待复原图像（图像历史的最后一张）。"""
        return self.trajectory.image_history[-1]
    def _stats(self, image: Image.Image) -> Optional[ImageStatistics]:
        return compute_image_statistics(image) if self.config.use_statistics_prior else None
    def _hint(self) -> Optional[List[str]]:
        return list(self.trajectory.sample.degradation_types) if self.config.provide_degradation_hint else None
    def _remaining(self) -> Optional[List[str]]:
        hint = self._hint()
        if hint is None:
            return None
        return [d for d in hint if d not in self._processed]
    def _state(self, stats: Optional[ImageStatistics]) -> AgentState:
        return AgentState(
            turn=self._turn,
            current_image=self.current_image,
            statistics=stats,
            degradation_hint=self._hint(),
            processed=list(self._processed),
            remaining=self._remaining(),
            tool_history=list(self.trajectory.executed_tools),
            retries_used=self._retries,
            last_error=self._last_error,
        )
```

```
    # ---------------------------------------------------------------- 交互
    def reset(self, sample: Sample) -> Observation:
        """载入样本并返回首轮观测。"""
        self._traj = Trajectory(sample=sample, mode=self.config.mode, image_history=[sample.degraded.convert("RG
B")])
        self._processed = []
        self._retries = 0
        self._turn = 0
        self._last_error = None
        stats = self._stats(self.current_image)
        user_text = build_first_user_prompt(self._hint(), stats.to_prompt_text() if stats else None, self.config
.mode)
        self._messages = [
            {"role": "system", "content": self.system_prompt},
            {"role": "user", "content": _text_image_content(user_text, self.current_image)},
        ]
        return Observation(messages=list(self._messages), state=self._state(stats))
    def _apply_tool_chain(self, calls: List[ToolCall], base_image: Image.Image):
        """依次执行工具链，返回 (最终图像, 已执行工具名, 代价, 错误)。"""
        image = base_image
        executed: List[str] = []
        cost = 0.0
        for call in calls:
            if not self.toolbox.has(call.name):
                return image, executed, cost, f"unknown tool '{call.name}'"
            try:
                result = self.toolbox.execute(call.name, image, call.arguments)
            except ToolExecutionError as exc:
                return image, executed, cost, str(exc)
            image = result.image
            executed.append(call.name)
            cost += self.toolbox.spec(call.name).cost
        return image, executed, cost, None
    def step(self, response: str) -> tuple[Observation, bool, StepRecord]:
        """执行一轮模型响应，返回（下一观测，是否终止，本步记录）。"""
        traj = self.trajectory
        self._turn += 1
        parsed: ParsedResponse = parse_response(response)
        idx_before = len(traj.image_history) - 1
        record = StepRecord(
            turn=self._turn, response=response, tool_calls=list(parsed.tool_calls),
            predicted_degradations=[c.degradation or degradation_of_tool(c.name) or "" for c in parsed.tool_call
s],
            executed_tools=[], image_index_before=idx_before, image_index_after=idx_before,
            format_errors=list(parsed.errors), is_answer=parsed.is_answer,
        )
        self._last_error = None
        # 终止：模型给出最终答案
        if parsed.is_answer and not parsed.errors:
            traj.answer = parsed.answer
```

```
            traj.finished = True
            traj.steps.append(record)
            return Observation(messages=list(self._messages), state=self._state(None)), True, record
        done = False
        calls = list(parsed.tool_calls)
        if self.config.max_tools_per_turn > 0 and len(calls) > self.config.max_tools_per_turn:
            record.format_errors.append(f"too many tools in one turn ({len(calls)} > {self.config.max_tools_per_
    turn})")
            calls = calls[: self.config.max_tools_per_turn]
        if not calls:
            self._last_error = "no valid tool call"
            if self.config.stop_on_format_error:
                done = True
        else:
            # 选择工具链的起始图像：多工具模式从原图；单工具模式从当前图，回退时从上一张图
            if self.config.mode == "multi_tool_planning":
                base = traj.image_history[0]
            elif calls[0].rollback and self._retries < self.config.max_retries and len(traj.image_history) >= 2:
                base = traj.image_history[-2]
                self._retries += 1
                if self._processed:
                    self._processed.pop()  # 回退撤销上一步的「已处理」记录
            else:
                base = self.current_image
            image, executed, cost, error = self._apply_tool_chain(calls, base)
            record.tool_cost = cost
            if error:
                record.error = error
                self._last_error = error
            if executed:
                traj.image_history.append(image)
                record.image_index_after = len(traj.image_history) - 1
                record.executed_tools = executed
                for name in executed:
                    deg = degradation_of_tool(name)
                    if deg and deg not in self._processed:
                        self._processed.append(deg)
        traj.steps.append(record)
        if self._turn >= self.config.max_turns:
            done = True
            traj.truncated = True
        # 记录助手回复并构造下一状态
        self._messages.append({"role": "assistant", "content": response})
        stats = self._stats(self.current_image)
        feedback = build_feedback_prompt(
            self.config.mode, record.executed_tools, self._processed, self._remaining(),
            stats.to_prompt_text() if stats else None, error=self._last_error,
        )
        self._messages.append({"role": "user", "content": _text_image_content(feedback, self.current_image)})
        return Observation(messages=list(self._messages), state=self._state(stats)), done, record
```

```
# -*- coding: utf-8 -*-
"""
策略接口与两种内置策略。
Policy.act(observation) -> 响应文本（严格遵循 <think>/<tool_call>/<answer> 格式）。
    OpenAICompatiblePolicy：调用 vLLM 等 OpenAI 兼容接口部署的多模态大模型（训练后的智能体）
    HeuristicPolicy       ：基于退化提示 + 图像统计先验的规则策略，用于基线、离线测试与数据流程验证
"""
from __future__ import annotations
import base64
import io
from abc import ABC, abstractmethod
from typing import Any, Dict, List, Optional, Sequence
from PIL import Image
from ..priors.image_statistics import ImageStatistics
from ..tools.registry import ToolSpec, tools_for_degradation
from .env import Observation
from .parser import ToolCall, format_answer, format_tool_call
# 默认处理优先级：先去除遮挡/几何类退化，再处理光照与噪声，最后细节恢复
DEFAULT_PRIORITY: Sequence[str] = (
    "rain", "haze", "motion blur", "defocus blur", "dark", "noise", "jpeg compression artifact", "low resolution
    ",
)
class Policy(ABC):
    """策略接口：根据观测输出符合格式的响应文本。"""
    @abstractmethod
    def act(self, observation: Observation) -> str:
        """根据观测生成一轮响应。"""
        ...
def _image_to_data_url(image: Image.Image) -> str:
    buf = io.BytesIO()
    image.convert("RGB").save(buf, format="PNG")
    return "" + base64.b64encode(buf.getvalue()).decode("ascii")
def messages_to_openai(messages: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """把环境消息（含 PIL 图像）转换为 OpenAI 兼容的多模态消息。"""
    out: List[Dict[str, Any]] = []
    for m in messages:
        content = m["content"]
        if isinstance(content, str):
            out.append({"role": m["role"], "content": content})
            continue
        parts: List[Dict[str, Any]] = []
        for c in content:
            if c["type"] == "text":
                parts.append({"type": "text", "text": c["text"]})
            else:
                parts.append({"type": "image_url", "image_url": {"url": _image_to_data_url(c["image"])}})
        out.append({"role": m["role"], "content": parts})
    return out
class OpenAICompatiblePolicy(Policy):
    """通过 OpenAI 兼容接口调用多模态大模型的策略。"""
```

```
    def __init__(self, base_url: str = "http://localhost:8000/v1", api_key: str = "EMPTY",
                 model: Optional[str] = None, temperature: float = 0.7, top_p: float = 0.9,
                 max_tokens: int = 2048, client: Any = None):
        if client is None:
            from openai import OpenAI  # 延迟导入，离线环境不依赖
            client = OpenAI(base_url=base_url, api_key=api_key)
        self.client = client
        self.model = model or self._detect_model()
        self.temperature, self.top_p, self.max_tokens = temperature, top_p, max_tokens
    def _detect_model(self) -> str:
        try:
            return self.client.models.list().data[0].id
        except Exception:  # noqa: BLE001
            return "default"
    def act(self, observation: Observation) -> str:
        """把观测转换为多模态消息并请求模型生成响应。"""
        resp = self.client.chat.completions.create(
            model=self.model, messages=messages_to_openai(observation.messages),
            temperature=self.temperature, top_p=self.top_p, max_tokens=self.max_tokens,
        )
        return resp.choices[0].message.content or ""
def select_tool_by_statistics(candidates: Sequence[ToolSpec], stats: Optional[ImageStatistics]) -> ToolSpec:
    """
    同类候选工具间的统计先验选择规则：
    边缘密度 / 高频能量高 → 偏细节保持型工具（描述含 detail / texture / fine）；
    否则 → 偏稳健平滑型工具（候选列表中的首个）。
    """
    if len(candidates) == 1 or stats is None:
        return candidates[0]
    q = stats.to_qualitative()
    detail_rich = q["edge_density"] == "high" or q["high_frequency_energy"] == "high"
    for spec in candidates:
        is_detail_tool = any(k in spec.description.lower() for k in ("detail", "texture", "fine"))
        if detail_rich and is_detail_tool:
            return spec
    return candidates[0]
class HeuristicPolicy(Policy):
    """规则策略：按优先级处理剩余退化，同类工具依据统计先验选择。"""
    def __init__(self, priority: Sequence[str] = DEFAULT_PRIORITY, mode: str = "single_tool_iterative",
                 visible_only: bool = True):
        self.priority = list(priority)
        self.mode = mode
        self.visible_only = visible_only
    def _ordered_remaining(self, remaining: Sequence[str]) -> List[str]:
        return sorted(remaining, key=lambda d: self.priority.index(d) if d in self.priority else len(self.priori
    ty))
    def _call_for(self, degradation: str, stats: Optional[ImageStatistics]) -> Optional[ToolCall]:
        candidates = tools_for_degradation(degradation, visible_only=self.visible_only)
        if not candidates:
            return None
```

```
        spec = select_tool_by_statistics(candidates, stats)
        return ToolCall(name=spec.name, degradation=degradation, arguments=spec.normalize_arguments({}))
    def act(self, observation: Observation) -> str:
        """按优先级处理剩余退化，同类工具依据统计先验选择。"""
        st = observation.state
        remaining = st.remaining if st.remaining is not None else []
        ordered = self._ordered_remaining(remaining)
        if not ordered:
            return format_answer("All hinted degradations have been processed; the image looks clean.", st.proce
    ssed)
        if self.mode == "multi_tool_planning":
            calls = [c for c in (self._call_for(d, st.statistics) for d in ordered) if c]
            return format_tool_call("Plan the full sequence by priority using image statistics.", calls)
        call = self._call_for(ordered[0], st.statistics)
        if call is None:
            return format_answer("No tool available for remaining degradations.", st.processed)
        why = f"Remaining: {', '.join(ordered)}. Address '{ordered[0]}' first with {call.name} based on image st
    atistics."
        return format_tool_call(why, [call])
# -*- coding: utf-8 -*-
"""轨迹采样：策略与环境交互直至终止。"""
from __future__ import annotations
from typing import Callable, Optional
from ..data.dataset import Sample
from .env import RestorationEnv
from .policy import Policy
from .trajectory import Trajectory
def run_episode(env: RestorationEnv, policy: Policy, sample: Sample,
                on_step: Optional[Callable[[int, str], None]] = None) -> Trajectory:
    """策略与环境交互直至终止，返回完整轨迹。"""
    obs = env.reset(sample)
    done = False
    while not done:
        response = policy.act(obs)
        obs, done, record = env.step(response)
        if on_step:
            on_step(record.turn, response)
    return env.trajectory
# -*- coding: utf-8 -*-
"""奖励函数：格式奖励、图像质量奖励、退化推理奖励、复合奖励。"""
from .format_reward import FormatConfig, check_format_enhanced, check_format_progressive
from .quality_metrics import psnr, ssim, lpips_distance, normalize_metrics, quality_score, no_reference_score
from .degradation_reward import degradation_set_reward, degradation_type_match, restoration_order_reward, is_cle
    an_answer
from .composite import RewardConfig, RewardBreakdown, compute_step_rewards, compute_episode_reward
__all__ = [
    "FormatConfig", "check_format_enhanced", "check_format_progressive",
    "psnr", "ssim", "lpips_distance", "normalize_metrics", "quality_score", "no_reference_score",
    "degradation_set_reward", "degradation_type_match", "restoration_order_reward", "is_clean_answer",
    "RewardConfig", "RewardBreakdown", "compute_step_rewards", "compute_episode_reward",
```

```
]
# -*- coding: utf-8 -*-
"""
格式奖励。
enhanced ：严格二元（1.0 / -1.0），用于模型已掌握基本格式后的强化阶段
progressive: 阶梯式（0.0 → 0.2 → 0.4 → 0.6 → 0.8 → 1.0），用于训练初期温和引导
输入为整条轨迹拼接后的多轮响应文本。
"""
from __future__ import annotations
import json
from dataclasses import dataclass, field
from typing import List, Optional, Sequence, Tuple
from ..agent.parser import ANSWER_RE, THINK_RE, TOOL_CALL_RE, parse_response, split_turns
from ..tools.registry import TOOL_SPECS
@dataclass
class FormatConfig:
    """格式检查配置：对话模式、单轮工具上限、总工具数上限与允许的工具集合。"""
    mode: str = "single_tool_iterative"
    max_tools_per_turn: int = 1
    enable_total_tools_upper_limit: bool = True   # 总工具数 <= 退化数 + 1
    min_think_chars: int = 10
    allowed_tools: Sequence[str] = field(default_factory=lambda: tuple(TOOL_SPECS.keys()))
def check_format_enhanced(text: str, degradation_count: int = 0, is_clean: bool = False,
                          cfg: Optional[FormatConfig] = None) -> Tuple[float, str]:
    """返回 (1.0, "ok") 或 (-1.0, 违规原因)。"""
    cfg = cfg or FormatConfig()
    turns = split_turns(text)
    if not turns:
        return -1.0, "no <think> block"
    total_tools = 0
    answer_turns: List[int] = []
    for i, turn in enumerate(turns, start=1):
        parsed = parse_response(turn)
        if parsed.think is None or len(parsed.think) < cfg.min_think_chars:
            return -1.0, f"turn {i}: think missing or too short"
        if parsed.errors:
            return -1.0, f"turn {i}: {parsed.errors[0]}"
        if parsed.is_answer:
            answer_turns.append(i)
            if set(parsed.answer.keys()) != {"restoration_log"}:
                return -1.0, f"turn {i}: answer must contain only restoration_log"
            continue
        n = len(parsed.tool_calls)
        if cfg.mode == "single_tool_iterative" and n != 1:
            return -1.0, f"turn {i}: single-tool mode requires exactly 1 tool, got {n}"
        if cfg.max_tools_per_turn > 0 and n > cfg.max_tools_per_turn:
            return -1.0, f"turn {i}: {n} tools exceeds per-turn limit {cfg.max_tools_per_turn}"
        for call in parsed.tool_calls:
            if call.name not in cfg.allowed_tools:
                return -1.0, f"turn {i}: tool '{call.name}' not allowed"
```

```
        total_tools += n
    if answer_turns and (len(answer_turns) > 1 or answer_turns[0] != len(turns)):
        return -1.0, "answer must appear only in the final turn"
    if not is_clean and total_tools < 1:
        return -1.0, "degraded sample requires at least one tool call"
    if cfg.enable_total_tools_upper_limit and not is_clean and degradation_count > 0 and total_tools > degradati
    on_count + 1:
        return -1.0, f"{total_tools} tool calls exceed limit {degradation_count + 1}"
    return 1.0, "ok"
def check_format_progressive(text: str, degradation_count: int = 0, is_clean: bool = False,
                             cfg: Optional[FormatConfig] = None) -> Tuple[float, str]:
    """
    阶梯式格式奖励：
        0.0 完全无格式 | 0.2 有 think | 0.4 有 tool_call 或 answer 标签
        0.6 JSON 可解析 | 0.8 工具名合法 | 1.0 通过全部增强检查
    """
    cfg = cfg or FormatConfig()
    if not THINK_RE.search(text):
        return 0.0, "level0: no think"
    if not (TOOL_CALL_RE.search(text) or ANSWER_RE.search(text)):
        return 0.2, "level1: think only"
    try:
        for m in TOOL_CALL_RE.finditer(text):
            json.loads(m.group(1))
        for m in ANSWER_RE.finditer(text):
            json.loads(m.group(1))
    except json.JSONDecodeError:
        return 0.4, "level2: tags present but json invalid"
    for turn in split_turns(text):
        parsed = parse_response(turn)
        for call in parsed.tool_calls:
            if call.name not in cfg.allowed_tools:
                return 0.6, f"level3: illegal tool {call.name}"
    score, reason = check_format_enhanced(text, degradation_count, is_clean, cfg)
    return (1.0, "level5: perfect") if score > 0 else (0.8, f"level4: {reason}")
# -*- coding: utf-8 -*-
"""
图像质量指标与归一化。
有参考：PSNR、SSIM、LPIPS（LPIPS 依赖可选，缺失时自动退化为 PSNR+SSIM）
无参考：基于统计先验的轻量质量分，可通过 ``register_no_reference_metric`` 挂接 NIQE/CLIP-IQA 等外部指标
归一化：PSNR 非线性映射 sqrt((psnr-10)/30)，SSIM 线性 (ssim+1)/2，LPIPS 直接裁剪到 [0,1]
"""
from __future__ import annotations
import math
from typing import Callable, Dict, Optional, Tuple
import cv2
import numpy as np
from PIL import Image
from ..priors.image_statistics import compute_image_statistics
_lpips_model = None
```

```
_no_reference_metric: Optional[Callable[[np.ndarray], float]] = None
def _to_array(img) -> np.ndarray:
    if isinstance(img, Image.Image):
        return np.array(img.convert("RGB"))
    arr = np.asarray(img)
    if arr.ndim == 2:
        arr = np.stack([arr] * 3, axis=-1)
    return arr[..., :3].astype(np.uint8)
def _match_size(img: np.ndarray, ref: np.ndarray) -> np.ndarray:
    """工具（如超分）可能改变尺寸，统一缩放到参考图尺寸再计算指标。"""
    if img.shape[:2] != ref.shape[:2]:
        img = cv2.resize(img, (ref.shape[1], ref.shape[0]), interpolation=cv2.INTER_AREA)
    return img
def psnr(img, ref) -> float:
    """峰值信噪比（dB），尺寸不一致时先缩放到参考图尺寸。"""
    a, b = _to_array(img), _to_array(ref)
    a = _match_size(a, b).astype(np.float64)
    mse = float(np.mean((a - b.astype(np.float64)) ** 2))
    if mse <= 1e-10:
        return 100.0
    return 10.0 * math.log10(255.0 ** 2 / mse)
def ssim(img, ref) -> float:
    """灰度 SSIM（高斯窗 11x11, sigma=1.5）。"""
    a, b = _to_array(img), _to_array(ref)
    a = _match_size(a, b)
    x = cv2.cvtColor(a, cv2.COLOR_RGB2GRAY).astype(np.float64)
    y = cv2.cvtColor(b, cv2.COLOR_RGB2GRAY).astype(np.float64)
    c1, c2 = (0.01 * 255) ** 2, (0.03 * 255) ** 2
    mu_x = cv2.GaussianBlur(x, (11, 11), 1.5)
    mu_y = cv2.GaussianBlur(y, (11, 11), 1.5)
    sxx = cv2.GaussianBlur(x * x, (11, 11), 1.5) - mu_x ** 2
    syy = cv2.GaussianBlur(y * y, (11, 11), 1.5) - mu_y ** 2
    sxy = cv2.GaussianBlur(x * y, (11, 11), 1.5) - mu_x * mu_y
    num = (2 * mu_x * mu_y + c1) * (2 * sxy + c2)
    den = (mu_x ** 2 + mu_y ** 2 + c1) * (sxx + syy + c2)
    return float(np.mean(num / den))
def lpips_distance(img, ref, net: str = "alex") -> Optional[float]:
    """LPIPS 感知距离；依赖缺失或权重不可用时返回 None。"""
    global _lpips_model
    try:
        import lpips  # noqa: WPS433
        import torch
        if _lpips_model is None:
            _lpips_model = lpips.LPIPS(net=net, verbose=False)
            _lpips_model.eval()
        a, b = _to_array(img), _to_array(ref)
        a = _match_size(a, b)
        to_t = lambda z: torch.from_numpy(z).permute(2, 0, 1).float().unsqueeze(0) / 127.5 - 1.0  # noqa: E731
        with torch.no_grad():
            return float(_lpips_model(to_t(a), to_t(b)).item())
```

```
    except Exception:  # noqa: BLE001
        return None
def normalize_metrics(psnr_val: float, ssim_val: float, lpips_val: Optional[float],
                      psnr_range: Tuple[float, float] = (10.0, 40.0)) -> Tuple[float, float, Optional[float]]:
    """把 PSNR、SSIM、LPIPS 归一化到 [0,1]。"""
    lo, hi = psnr_range
    p = max(0.0, min(1.0, (psnr_val - lo) / (hi - lo)))
    p = math.sqrt(p)
    s = max(0.0, min(1.0, (ssim_val + 1.0) / 2.0))
    l = None if lpips_val is None else max(0.0, min(1.0, lpips_val))
    return p, s, l
def quality_score(img, ref, w_psnr: float = 0.3, w_ssim: float = 0.3, w_lpips: float = 0.4,
                  use_lpips: bool = False) -> Dict[str, float]:
    """
    复合质量分 R_quality ∈ [0,1] :
        λ_psnr·PSNR_norm + λ_ssim·SSIM_norm + λ_lpips·(1-LPIPS_norm)
    LPIPS 不可用时权重按比例重新分配给 PSNR 与 SSIM。
    """
    p_raw, s_raw = psnr(img, ref), ssim(img, ref)
    l_raw = lpips_distance(img, ref) if use_lpips else None
    p, s, l = normalize_metrics(p_raw, s_raw, l_raw)
    if l is None:
        total = w_psnr + w_ssim
        score = (w_psnr * p + w_ssim * s) / total if total > 0 else 0.0
    else:
        score = w_psnr * p + w_ssim * s + w_lpips * (1.0 - l)
    return {"psnr": p_raw, "ssim": s_raw, "lpips": l_raw if l_raw is not None else float("nan"),
            "score": float(max(0.0, min(1.0, score)))}
def register_no_reference_metric(fn: Optional[Callable[[np.ndarray], float]]) -> None:
    """挂接外部无参考指标（输入 RGB uint8 数组，输出 [0,1] 分数，越大越好）。"""
    global _no_reference_metric
    _no_reference_metric = fn
def no_reference_score(img) -> float:
    """
    无参考质量分 [0,1]。若注册了外部指标则直接使用；否则由统计先验给出轻量代理：
    清晰度（拉普拉斯方差）↑、噪声估计↓、对比度适中、块效应↓、雾浓度↓。
    """
    arr = _to_array(img)
    if _no_reference_metric is not None:
        return float(max(0.0, min(1.0, _no_reference_metric(arr))))
    st = compute_image_statistics(arr)
    sharp = min(1.0, st.blur_score / 400.0)
    noise = 1.0 - min(1.0, st.noise_sigma / 20.0)
    contrast = 1.0 - abs(st.contrast_global - 55.0) / 55.0
    block = 1.0 - min(1.0, st.blockiness / 0.08)
    haze = 1.0 - min(1.0, max(0.0, (st.haze_score - 0.2) / 0.5))
    return float(max(0.0, min(1.0, 0.3 * sharp + 0.25 * noise + 0.15 * max(0.0, contrast) + 0.15 * block + 0.15
    * haze)))
def discretize(score: float, levels: int) -> float:
    """把连续分数量化为 levels 档（0 表示不量化）。"""
```

```
    if levels <= 0:
        return score
    return math.floor(score * levels) / levels
# -*- coding: utf-8 -*-
"""退化推理奖励与复原顺序奖励。"""
from __future__ import annotations
from typing import Iterable, List, Sequence
def _clean_list(items: Iterable[str]) -> List[str]:
    return [str(x).strip().lower() for x in items if x is not None and str(x).strip().lower() != "clean"]
def merge_consecutive_duplicates(items: Sequence[str]) -> List[str]:
    """合并相邻的重复项。"""
    out: List[str] = []
    for x in items:
        if not out or out[-1] != x:
            out.append(x)
    return out
def degradation_set_reward(predicted: Iterable[str], truth: Iterable[str]) -> float:
    """
    退化推理奖励 R_deg = |D^ ∩ D| / max(1, |D^|) :
    预测集合中正确类型所占比例（惩罚误报，不惩罚漏报，漏报由后续步骤继续推理）。
    """
    p, t = set(_clean_list(predicted)), set(_clean_list(truth))
    if not p:
        return 0.0
    return len(p & t) / max(1, len(p))
def degradation_type_match(predicted_log: Iterable[str], truth: Iterable[str]) -> float:
    """
    退化类型集合匹配（不考虑顺序）：
    预测含真实集合之外的类型 → 0；完全匹配 → 1；否则为覆盖比例。
    """
    t = set(_clean_list(truth))
    if not t:
        return 0.0
    p = set(merge_consecutive_duplicates(_clean_list(predicted_log)))
    if not p or not p.issubset(t):
        return 0.0
    return 1.0 if p == t else len(p) / len(t)
def restoration_order_reward(predicted_log: Iterable[str], addition_order: Sequence[str]) -> float:
    """
    复原顺序奖励：理想复原顺序为退化施加顺序的逆序（后进先出）。
    完全一致 → 1；按位置匹配比例给部分分；集合正确但顺序错误至少 0.1。
    """
    pred = merge_consecutive_duplicates(_clean_list(predicted_log))
    ideal = list(reversed(_clean_list(addition_order)))
    if not pred or not ideal:
        return 0.0
    if pred == ideal:
        return 1.0
    correct = sum(1 for a, b in zip(pred, ideal) if a == b)
    score = correct / len(ideal)
```

```
    if set(pred) == set(ideal):
        score = max(score, 0.1)
    return score
def is_clean_answer(restoration_log: Iterable[str], num_tool_calls: int) -> bool:
    """clean 样本的正确响应：不调用工具，且 restoration_log 为空或仅含 clean。"""
    return num_tool_calls == 0 and len(_clean_list(restoration_log)) == 0
# -*- coding: utf-8 -*-
"""
复合奖励。
步级奖励（用于 PPO 等逐步信用分配）：
    r_t = α·ΔQ_t + β·R_deg(D^_t, D) - γ·Cost(a_t) - δ
    终止奖励 r_final = κ·R_quality(I_T)，未在最大轮数内完成则给失败惩罚
轨迹级奖励（用于 GRPO 等序列级优化，与 VeRL 训练路径一致）：
    R = w_fmt·format + w_q·quality [+ w_deg·type_match] [+ w_mid·intermediate] [+ diversity_bonus]
    格式违规时仅保留负的格式项，不计算其他奖励。
"""
from __future__ import annotations
from dataclasses import dataclass, field
from typing import Dict, List, Optional
from ..agent.trajectory import Trajectory
from .degradation_reward import degradation_set_reward, degradation_type_match, is_clean_answer
from .format_reward import FormatConfig, check_format_enhanced, check_format_progressive
from .quality_metrics import discretize, no_reference_score, quality_score
@dataclass
class RewardConfig:
    # 步级奖励系数
    """奖励配置：步级系数、轨迹级权重、质量指标与格式模式。"""
    alpha: float = 1.0
    beta: float = 0.5
    gamma: float = 0.05
    delta: float = 0.02
    kappa: float = 1.0
    failure_penalty: float = -0.5
    # 轨迹级奖励权重
    format_weight: float = 0.3
    quality_weight: float = 0.7
    enable_degradation_type_reward: bool = False
    degradation_type_weight: float = 1.0
    enable_intermediate_reward: bool = False
    intermediate_weight: float = 0.5
    tool_diversity_bonus: float = 0.1
    # 质量指标
    use_no_reference: bool = False
    use_lpips: bool = False
    discretize_levels: int = 0
    # 格式
    format_mode: str = "enhanced"          # enhanced | progressive
    format: FormatConfig = field(default_factory=FormatConfig)
@dataclass
class RewardBreakdown:
```

```
    """轨迹奖励的分项明细，便于训练监控。"""
    score: float
    format_score: float
    format_reason: str
    quality_score: float
    degradation_type_score: float = 0.0
    intermediate_quality_score: float = 0.0
    tool_diversity_bonus: float = 0.0
    clean_accuracy: Optional[float] = None
    metrics: Dict[str, float] = field(default_factory=dict)
    def to_dict(self) -> dict:
        """转换为扁平字典。"""
        d = {
            "score": self.score, "format_score": self.format_score, "format_reason": self.format_reason,
            "quality_score": self.quality_score, "degradation_type_score": self.degradation_type_score,
            "intermediate_quality_score": self.intermediate_quality_score,
            "tool_diversity_bonus": self.tool_diversity_bonus,
        }
        if self.clean_accuracy is not None:
            d["clean_accuracy"] = self.clean_accuracy
        d.update({f"metric_{k}": v for k, v in self.metrics.items()})
        return d
def _quality(image, reference, cfg: RewardConfig) -> Dict[str, float]:
    if cfg.use_no_reference or reference is None:
        return {"score": no_reference_score(image)}
    return quality_score(image, reference, use_lpips=cfg.use_lpips)
def compute_step_rewards(traj: Trajectory, cfg: Optional[RewardConfig] = None) -> List[float]:
    """按步计算 r_t；长度与 traj.steps 相同。"""
    cfg = cfg or RewardConfig()
    truth = traj.sample.degradation_types
    ref = traj.sample.clean
    q_cache: Dict[int, float] = {}
    def q(idx: int) -> float:
        if idx not in q_cache:
            q_cache[idx] = _quality(traj.image_history[idx], ref, cfg)["score"]
        return q_cache[idx]
    rewards: List[float] = []
    for step in traj.steps:
        if step.is_answer:
            r = cfg.kappa * q(step.image_index_after) if traj.finished else cfg.failure_penalty
            rewards.append(r)
            continue
        delta_q = q(step.image_index_after) - q(step.image_index_before)
        r_deg = degradation_set_reward(step.predicted_degradations, truth) if truth else 0.0
        r = cfg.alpha * delta_q + cfg.beta * r_deg - cfg.gamma * step.tool_cost - cfg.delta
        if step.format_errors or step.error:
            r -= 0.1
        rewards.append(r)
    if traj.truncated and not traj.finished and rewards:
        rewards[-1] += cfg.failure_penalty
```

```
    return rewards
def compute_episode_reward(traj: Trajectory, cfg: Optional[RewardConfig] = None) -> RewardBreakdown:
    """轨迹级复合奖励（GRPO 序列级）。"""
    cfg = cfg or RewardConfig()
    sample = traj.sample
    truth = sample.degradation_types
    is_clean = sample.is_clean
    fmt_cfg = FormatConfig(mode=traj.mode, max_tools_per_turn=cfg.format.max_tools_per_turn,
                           enable_total_tools_upper_limit=cfg.format.enable_total_tools_upper_limit,
                           allowed_tools=cfg.format.allowed_tools)
    checker = check_format_progressive if cfg.format_mode == "progressive" else check_format_enhanced
    fmt, reason = checker(traj.concatenated_response, len(truth), is_clean, fmt_cfg)
    if is_clean:
        acc = 1.0 if is_clean_answer(traj.restoration_log, traj.num_tool_calls) else 0.0
        score = cfg.format_weight * fmt + (cfg.quality_weight * acc if fmt > 0 else 0.0)
        return RewardBreakdown(score, fmt, reason, acc, clean_accuracy=acc)
    if fmt <= 0:
        return RewardBreakdown(cfg.format_weight * fmt, fmt, reason, 0.0)
    executed_any = len(traj.image_history) > 1
    metrics = _quality(traj.final_image, sample.clean, cfg) if executed_any else {"score": 0.0}
    q_final = discretize(metrics["score"], cfg.discretize_levels)
    score = cfg.format_weight * fmt + cfg.quality_weight * q_final
    deg_score = 0.0
    if cfg.enable_degradation_type_reward:
        predicted = traj.restoration_log or [d for s in traj.steps for d in s.predicted_degradations]
        deg_score = degradation_type_match(predicted, truth)
        score += cfg.degradation_type_weight * deg_score
    mid_score = 0.0
    if cfg.enable_intermediate_reward and len(traj.image_history) > 2:
        mids = traj.image_history[1:-1]
        mid_score = sum(no_reference_score(m) for m in mids) / len(mids)
        score += cfg.intermediate_weight * mid_score
    bonus = 0.0
    tools = traj.executed_tools
    if tools and len(set(tools)) == len(tools) and 1 <= len(tools) <= len(truth) + 1:
        bonus = cfg.tool_diversity_bonus
        score += bonus
    return RewardBreakdown(score, fmt, reason, q_final, deg_score, mid_score, bonus,
                           metrics={k: v for k, v in metrics.items() if k != "score"})
# -*- coding: utf-8 -*-
"""训练：GRPO 组相对优势、可训练工具选择策略、VeRL 框架桥接。"""
from .grpo import group_relative_advantages, GRPOConfig, GRPOTrainer, TrainablePolicy
from .softmax_policy import SoftmaxToolPolicy
from .verl_plugin import compute_score, make_verl_tool_classes
__all__ = [
    "group_relative_advantages", "GRPOConfig", "GRPOTrainer", "TrainablePolicy",
    "SoftmaxToolPolicy", "compute_score", "make_verl_tool_classes",
]
# -*- coding: utf-8 -*-
"""
```

```
组相对策略优化（GRPO）训练循环。
对每个训练样本采样 G 条轨迹，以组内奖励均值/标准差归一化得到优势
    A_i = (r_i - mean(r)) / (std(r) + ϵ)
再交由可训练策略按优势更新参数。该循环与具体策略实现解耦：
    - SoftmaxToolPolicy ：轻量表格化策略，可在 CPU 上完整运行（测试 / 原型）
    - 多模态大模型策略 ：由 VeRL 框架负责参数更新，本模块通过 verl_plugin 提供奖励函数
"""
from __future__ import annotations
import statistics
from abc import abstractmethod
from dataclasses import dataclass, field
from typing import Callable, Dict, List, Sequence
from ..agent.env import RestorationEnv
from ..agent.policy import Policy
from ..agent.runner import run_episode
from ..agent.trajectory import Trajectory
from ..data.dataset import Sample
from ..reward.composite import RewardConfig, compute_episode_reward
class TrainablePolicy(Policy):
    """可训练策略接口：在轨迹采样前后记录动作，并按优势更新。"""
    @abstractmethod
    def begin_episode(self) -> None:
        """开始记录一条轨迹的动作。"""
    @abstractmethod
    def end_episode(self) -> List[object]:
        """返回本轨迹中记录的动作（供 update 使用）。"""
    @abstractmethod
    def update(self, episodes: Sequence[List[object]], advantages: Sequence[float]) -> Dict[str, float]:
        """按组相对优势更新策略参数并返回统计。"""
def group_relative_advantages(rewards: Sequence[float], eps: float = 1e-6) -> List[float]:
    """计算组内标准化优势 (r-mean)/(std+eps)。"""
    if len(rewards) == 0:
        return []
    if len(rewards) == 1:
        return [0.0]
    mean = statistics.fmean(rewards)
    std = statistics.pstdev(rewards)
    return [(r - mean) / (std + eps) for r in rewards]
@dataclass
class GRPOConfig:
    """GRPO 配置：组大小、训练轮数与奖励配置。"""
    group_size: int = 4
    epochs: int = 1
    reward: RewardConfig = field(default_factory=RewardConfig)
@dataclass
class TrainStats:
    """单个训练步的统计量。"""
    step: int
    mean_reward: float
    mean_quality: float
```

```
    format_violation_rate: float
    mean_tool_calls: float
    policy_stats: Dict[str, float] = field(default_factory=dict)
class GRPOTrainer:
    """组相对策略优化训练循环，与具体策略实现解耦。"""
    def __init__(self, env_factory: Callable[[], RestorationEnv], policy: TrainablePolicy,
                 config: GRPOConfig | None = None):
        self.env_factory = env_factory
        self.policy = policy
        self.config = config or GRPOConfig()
        self.history: List[TrainStats] = []
    def rollout_group(self, sample: Sample) -> tuple[List[Trajectory], List[List[object]]]:
        """对同一样本采样一组轨迹。"""
        trajs, actions = [], []
        for _ in range(self.config.group_size):
            env = self.env_factory()
            self.policy.begin_episode()
            trajs.append(run_episode(env, self.policy, sample))
            actions.append(self.policy.end_episode())
        return trajs, actions
    def train_step(self, samples: Sequence[Sample]) -> TrainStats:
        """对一批样本完成采样、奖励、优势计算与策略更新。"""
        all_actions: List[List[object]] = []
        all_adv: List[float] = []
        rewards, qualities, violations, calls = [], [], [], []
        for sample in samples:
            trajs, actions = self.rollout_group(sample)
            breakdowns = [compute_episode_reward(t, self.config.reward) for t in trajs]
            group_rewards = [b.score for b in breakdowns]
            all_actions.extend(actions)
            all_adv.extend(group_relative_advantages(group_rewards))
            rewards.extend(group_rewards)
            qualities.extend(b.quality_score for b in breakdowns)
            violations.extend(1.0 if b.format_score <= 0 else 0.0 for b in breakdowns)
            calls.extend(float(t.num_tool_calls) for t in trajs)
        policy_stats = self.policy.update(all_actions, all_adv)
        stats = TrainStats(
            step=len(self.history) + 1,
            mean_reward=statistics.fmean(rewards) if rewards else 0.0,
            mean_quality=statistics.fmean(qualities) if qualities else 0.0,
            format_violation_rate=statistics.fmean(violations) if violations else 0.0,
            mean_tool_calls=statistics.fmean(calls) if calls else 0.0,
            policy_stats=policy_stats,
        )
        self.history.append(stats)
        return stats
    def fit(self, samples: Sequence[Sample], batch_size: int = 4,
            on_step: Callable[[TrainStats], None] | None = None) -> List[TrainStats]:
        """按批次遍历样本完成多轮训练，返回训练历史。"""
        for _ in range(self.config.epochs):
```

```
            for i in range(0, len(samples), batch_size):
                stats = self.train_step(samples[i:i + batch_size])
                if on_step:
                    on_step(stats)
        return self.history
# -*- coding: utf-8 -*-
"""
可训练的表格化工具选择策略。
状态键 = (待处理退化类型, 图像细节丰富度分桶)，动作 = 该退化类型下的候选工具。
策略参数为每个状态键上的工具 logits，按 softmax 采样；
更新采用策略梯度 ∇ log π(a|s)·A，即 GRPO 组相对优势下的 REINFORCE。
退化处理顺序遵循优先级表（与 HeuristicPolicy 一致），顺序决策可由大模型策略学习。
"""
from __future__ import annotations
import json
import math
import random
from dataclasses import dataclass
from pathlib import Path
from typing import Dict, List, Optional, Sequence, Tuple
from ..agent.env import Observation
from ..agent.parser import ToolCall, format_answer, format_tool_call
from ..agent.policy import DEFAULT_PRIORITY
from ..priors.image_statistics import ImageStatistics
from ..tools.registry import tools_for_degradation
from .grpo import TrainablePolicy
@dataclass
class ActionRecord:
    """训练用动作记录：状态键、所选工具下标与对数概率。"""
    key: Tuple[str, str]
    tool_index: int
    log_prob: float
def detail_bucket(stats: Optional[ImageStatistics]) -> str:
    """按边缘密度与高频能量把图像划分为细节丰富、平滑或中等。"""
    if stats is None:
        return "unknown"
    q = stats.to_qualitative()
    if q["edge_density"] == "high" or q["high_frequency_energy"] == "high":
        return "detail_rich"
    if q["edge_density"] == "low":
        return "smooth"
    return "medium"
class SoftmaxToolPolicy(TrainablePolicy):
    """表格化 softmax 工具选择策略，按策略梯度更新。"""
    def __init__(self, lr: float = 0.5, temperature: float = 1.0, seed: int = 0,
                 priority: Sequence[str] = DEFAULT_PRIORITY, visible_only: bool = False):
        self.lr = lr
        self.temperature = temperature
        self.rng = random.Random(seed)
        self.priority = list(priority)
```

```
        self.visible_only = visible_only
        self.logits: Dict[Tuple[str, str], List[float]] = {}
        self._episode: List[ActionRecord] = []
    # ------------------------------------------------------------ 参数
    def candidates(self, degradation: str) -> List[str]:
        """返回处理指定退化类型的候选工具名。"""
        return [s.name for s in tools_for_degradation(degradation, visible_only=self.visible_only)]
    def _logits(self, key: Tuple[str, str]) -> List[float]:
        if key not in self.logits:
            self.logits[key] = [0.0] * len(self.candidates(key[0]))
        return self.logits[key]
    def probabilities(self, degradation: str, bucket: str) -> Dict[str, float]:
        """返回状态键下各候选工具的选择概率。"""
        logits = self._logits((degradation, bucket))
        m = max(logits)
        exp = [math.exp((z - m) / self.temperature) for z in logits]
        total = sum(exp)
        return {name: e / total for name, e in zip(self.candidates(degradation), exp)}
    # ------------------------------------------------------------ 交互
    def begin_episode(self) -> None:
        """清空本轨迹的动作记录。"""
        self._episode = []
    def end_episode(self) -> List[ActionRecord]:
        """返回并清空本轨迹记录的动作。"""
        ep, self._episode = self._episode, []
        return ep
    def act(self, observation: Observation) -> str:
        """按当前策略采样工具并生成响应。"""
        st = observation.state
        remaining = st.remaining if st.remaining is not None else []
        ordered = sorted(remaining, key=lambda d: self.priority.index(d) if d in self.priority else 99)
        if not ordered:
            return format_answer("All hinted degradations have been processed.", st.processed)
        deg = ordered[0]
        names = self.candidates(deg)
        if not names:
            return format_answer("No tool available.", st.processed)
        bucket = detail_bucket(st.statistics)
        probs = self.probabilities(deg, bucket)
        r, acc, idx = self.rng.random(), 0.0, len(names) - 1
        for i, name in enumerate(names):
            acc += probs[name]
            if r <= acc:
                idx = i
                break
        self._episode.append(ActionRecord((deg, bucket), idx, math.log(max(probs[names[idx]], 1e-12))))
        call = ToolCall(name=names[idx], degradation=deg, arguments={})
        return format_tool_call(f"Address '{deg}' (bucket={bucket}) with {names[idx]} sampled from learned polic
y.", [call])
    # ------------------------------------------------------------ 更新
```

```
    def update(self, episodes: Sequence[List[ActionRecord]], advantages: Sequence[float]) -> Dict[str, float]:
        """按优势对 logits 做策略梯度更新。"""
        total_grad = 0.0
        n = 0
        for records, adv in zip(episodes, advantages):
            if adv == 0.0:
                continue
            for rec in records:
                logits = self._logits(rec.key)
                probs = list(self.probabilities(*rec.key).values())
                for i in range(len(logits)):
                    grad = (1.0 if i == rec.tool_index else 0.0) - probs[i]
                    logits[i] += self.lr * adv * grad
                    total_grad += abs(grad * adv)
                n += 1
        return {"num_actions": float(n), "mean_abs_grad": total_grad / n if n else 0.0}
    # ---------------------------------------------------------- 持久化
    def save(self, path: str | Path) -> None:
        """把策略参数保存为 JSON。"""
        data = {f"{k[0]}|{k[1]}": v for k, v in self.logits.items()}
        Path(path).write_text(json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")
    def load(self, path: str | Path) -> None:
        """从 JSON 载入策略参数。"""
        data = json.loads(Path(path).read_text(encoding="utf-8"))
        self.logits = {tuple(k.split("|", 1)): v for k, v in data.items()}  # type: ignore[misc]
# -*- coding: utf-8 -*-
"""
VeRL 智能体强化学习框架桥接（框架本身为独立的开源项目，本软件不包含其源码）。
1. compute_score(...)：序列级奖励函数；verl_compute_score(...) 是适配框架奖励管理器
  (data_source, solution_str, ground_truth, extra_info) 调用约定的入口。输入为多轮拼接的模型输出、ground_truth
   （含 reward_model 与 env_name）以及 extra_info（含原图与工具处理后的图像历史）。
2. make_verl_tool_classes(ToolBase)：把注册表中的每个工具动态生成为 VeRL ToolBase
  子类（name 属性对应训练数据 env_name），execute 内部复用本包 Toolbox。
"""
from __future__ import annotations
import io
import json
import os
from typing import Any, Dict, List, Optional, Sequence, Union
from PIL import Image
from ..agent.parser import parse_response, split_turns
from ..reward.degradation_reward import degradation_type_match, is_clean_answer
from ..reward.format_reward import FormatConfig, check_format_enhanced, check_format_progressive
from ..reward.quality_metrics import discretize, no_reference_score, quality_score
from ..tools.registry import TOOL_SPECS, degradation_of_tool
from ..tools.toolbox import Toolbox, build_toolbox
def _env_flag(name: str, default: bool) -> bool:
    return os.environ.get(name, str(default)).strip().lower() in ("1", "true", "yes")
def _env_float(name: str, default: float) -> float:
    try:
```

```
        return float(os.environ.get(name, default))
    except ValueError:
        return default
def _to_image(data: Any) -> Optional[Image.Image]:
    if data is None:
        return None
    if isinstance(data, Image.Image):
        return data.convert("RGB")
    if isinstance(data, dict):
        if "image" in data and isinstance(data["image"], list) and data["image"]:
            return _to_image(data["image"][0])
        if "bytes" in data:
            return Image.open(io.BytesIO(data["bytes"])).convert("RGB")
    if isinstance(data, (bytes, bytearray)):
        return Image.open(io.BytesIO(data)).convert("RGB")
    if isinstance(data, str) and os.path.exists(data):
        return Image.open(data).convert("RGB")
    return None
def _image_history(extra_info: Optional[Dict[str, Any]]) -> List[Image.Image]:
    if not extra_info:
        return []
    hist = extra_info.get("image_history") or []
    images = [_to_image(h) for h in hist]
    return [im for im in images if im is not None]
def compute_score(solution_str: str, ground_truth: Union[str, Dict[str, Any]],
                  extra_info: Optional[Dict[str, Any]] = None, **overrides: Any) -> Dict[str, Any]:
    """
    VeRL 序列级奖励。配置优先级：显式关键字参数 > 环境变量 > 默认值。
    环境变量：FORMAT_REWARD_WEIGHT, QUALITY_REWARD_WEIGHT, ENABLE_DEGRADATION_TYPE_REWARD,
          DEGRADATION_TYPE_REWARD_WEIGHT, ENABLE_INTERMEDIATE_REWARD, INTERMEDIATE_REWARD_WEIGHT,
          IMAGE_QUALITY_USE_NO_REFERENCE, IMAGE_QUALITY_DISCRETIZE_LEVELS, USE_PROGRESSIVE_FORMAT,
          AGENT_CONVERSATION_MODE, MAX_TOOLS_PER_TURN, ENABLE_TOTAL_TOOLS_UPPER_LIMIT
    """
    if isinstance(ground_truth, str):
        try:
            ground_truth = json.loads(ground_truth)
        except json.JSONDecodeError:
            ground_truth = {}
    ground_truth = ground_truth or {}
    reward_model = ground_truth.get("reward_model") or []
    env_name = str(ground_truth.get("env_name", "") or "")
    truth = [d.get("degradation_type") for d in reward_model if isinstance(d, dict)] or \
        [x.strip() for x in env_name.split(",") if x.strip() and x.strip().lower() != "clean"]
    is_clean = env_name.strip().lower() == "clean" or (not truth and not reward_model)
    cfg = {
        "format_weight": overrides.get("format_weight", _env_float("FORMAT_REWARD_WEIGHT", 0.3)),
        "quality_weight": overrides.get("quality_weight", _env_float("QUALITY_REWARD_WEIGHT", 0.7)),
        "enable_degradation_type_reward": overrides.get("enable_degradation_type_reward", _env_flag("ENABLE_DEGR
    ADATION_TYPE_REWARD", False)),
        "degradation_type_weight": overrides.get("degradation_type_weight", _env_float("DEGRADATION_TYPE_REWARD_
```

```
WEIGHT", 1.0)),
    "enable_intermediate_reward": overrides.get("enable_intermediate_reward", _env_flag("ENABLE_INTERMEDIATE
_REWARD", False)),
    "intermediate_weight": overrides.get("intermediate_weight", _env_float("INTERMEDIATE_REWARD_WEIGHT", 0.5
)),
    "use_no_reference": overrides.get("use_no_reference", _env_flag("IMAGE_QUALITY_USE_NO_REFERENCE", False)
),
    "discretize_levels": int(overrides.get("discretize_levels", _env_float("IMAGE_QUALITY_DISCRETIZE_LEVELS"
, 0))),
    "progressive": overrides.get("progressive", _env_flag("USE_PROGRESSIVE_FORMAT", False)),
    "mode": overrides.get("mode", os.environ.get("AGENT_CONVERSATION_MODE", "single_tool_iterative")),
    "max_tools_per_turn": int(overrides.get("max_tools_per_turn", _env_float("MAX_TOOLS_PER_TURN", 1))),
    "upper_limit": overrides.get("upper_limit", _env_flag("ENABLE_TOTAL_TOOLS_UPPER_LIMIT", True)),
}
fmt_cfg = FormatConfig(mode=cfg["mode"], max_tools_per_turn=cfg["max_tools_per_turn"],
                       enable_total_tools_upper_limit=cfg["upper_limit"])
checker = check_format_progressive if cfg["progressive"] else check_format_enhanced
fmt, reason = checker(solution_str, len(truth), is_clean, fmt_cfg)
turns = [parse_response(t) for t in split_turns(solution_str)]
num_calls = sum(len(t.tool_calls) for t in turns)
answer = next((t.answer for t in reversed(turns) if t.answer), None)
restoration_log = [str(x) for x in (answer or {}).get("restoration_log", [])] if answer else []
result: Dict[str, Any] = {
    "format_score": fmt, "format_reason": reason, "quality_score": 0.0,
    "degradation_type_score": 0.0, "intermediate_quality_score": 0.0, "tool_diversity_bonus": 0.0,
    "is_clean_sample": 1.0 if is_clean else 0.0,
    "degradation_types_all": ", ".join(truth) if truth else "clean",
}
if is_clean:
    acc = 1.0 if is_clean_answer(restoration_log, num_calls) else 0.0
    result["clean_accuracy"] = acc
    result["score"] = cfg["format_weight"] * fmt + (cfg["quality_weight"] * acc if fmt > 0 else 0.0)
    return result
if fmt <= 0:
    result["score"] = cfg["format_weight"] * fmt
    return result
history = _image_history(extra_info)
original = _to_image((extra_info or {}).get("original_image"))
quality = 0.0
if len(history) >= 2:  # history[0] 为退化输入，之后为工具输出
    final = history[-1]
    if cfg["use_no_reference"] or original is None:
        quality = no_reference_score(final)
    else:
        quality = quality_score(final, original)["score"]
    quality = discretize(quality, cfg["discretize_levels"])
result["quality_score"] = quality
score = cfg["format_weight"] * fmt + cfg["quality_weight"] * quality
if cfg["enable_degradation_type_reward"]:
    predicted = restoration_log or [c.degradation or degradation_of_tool(c.name) or "" for t in turns for c
```

```
    in t.tool_calls]
        result["degradation_type_score"] = degradation_type_match(predicted, truth)
        score += cfg["degradation_type_weight"] * result["degradation_type_score"]
    if cfg["enable_intermediate_reward"] and len(history) > 2:
        mids = history[1:-1]
        result["intermediate_quality_score"] = sum(no_reference_score(m) for m in mids) / len(mids)
        score += cfg["intermediate_weight"] * result["intermediate_quality_score"]
    tools = [c.name for t in turns for c in t.tool_calls]
    if tools and len(set(tools)) == len(tools) and len(tools) <= len(truth) + 1:
        result["tool_diversity_bonus"] = 0.1
        score += 0.1
    result["score"] = score
    return result
def verl_compute_score(data_source: str, solution_str: str, ground_truth: Union[str, Dict[str, Any]],
                       extra_info: Optional[Dict[str, Any]] = None) -> Dict[str, Any]:
    """适配框架奖励管理器的调用约定，在框架的 default_compute_score 中按 data_source 分派到这里。"""
    return compute_score(solution_str, ground_truth, extra_info)
def make_verl_tool_classes(tool_base: type, toolbox: Optional[Toolbox] = None,
                           tool_names: Optional[Sequence[str]] = None) -> Dict[str, type]:
    """
    为 VeRL 的 ToolBase 注册表动态生成工具类。
    生成类遵循框架的工具约定：类属性 name；reset(raw_prompt, multi_modal_data, origin_multi_modal_data)；
    execute(action_string) -> (obs, reward, done, info)，obs 含 prompt 与 multi_modal_data。
    """
    toolbox = toolbox or build_toolbox("remote")
    names = list(tool_names) if tool_names else list(TOOL_SPECS.keys())
    classes: Dict[str, type] = {}
    user_prompt = ("The `{tool}` tool was applied successfully. Analyse the new image for any remaining "
                   "degradations and continue.")
    for name in names:
        def _reset(self, raw_prompt, multi_modal_data, origin_multi_modal_data=None, **kwargs):
            self.chatml_history = raw_prompt
            self.multi_modal_data = multi_modal_data or origin_multi_modal_data
        def _execute(self, action_string: str, _tool_name: str = name, **kwargs):
            parsed = parse_response(action_string)
            if parsed.is_answer:
                return "", 0.0, True, {"status": "finished", "answer": parsed.answer}
            calls = [c for c in parsed.tool_calls if c.name == _tool_name]
            if not calls:
                obs = f"\n<|im_start|>user\nError: no valid call for {_tool_name}<|im_end|>\n<|im_start|>assista
    nt\n"
                return obs, -0.1, False, {"status": "failed", "error": "no tool call"}
            try:
                image = self.multi_modal_data["image"][0]
                result = toolbox.execute(_tool_name, image, calls[0].arguments)
            except Exception as exc:  # noqa: BLE001
                obs = f"\n<|im_start|>user\nError: {exc}<|im_end|>\n<|im_start|>assistant\n"
                return obs, -0.1, False, {"status": "failed", "error": str(exc)}
            self.multi_modal_data["image"][0] = result.image
            obs = {
```

```
            "prompt": "\n<|im_start|>user\n<tool_response><image>" + user_prompt.format(tool=_tool_name)
                    + "</tool_response><|im_end|>\n<|im_start|>assistant\n",
            "multi_modal_data": {"image": [result.image]},
        }
        return obs, 0.0, False, {"status": "success", "tool_used": _tool_name, "execution_time": result.elap
    sed}
    def _init(self, _name=None, _desc="", _params=None, **kwargs):
        tool_base.__init__(self, name=self.name, description=_desc, parameters=_params or {})
        self.chatml_history = []
        self.multi_modal_data = None
    cls = type(f"RestoraMind_{name}", (tool_base,), {
        "name": name, "__init__": _init, "reset": _reset, "execute": _execute,
    })
    classes[name] = cls
  return classes
# -*- coding: utf-8 -*-
"""推理：单图复原与数据集评估。"""
from .restore import RestoreResult, restore_image
from .evaluate import evaluate_samples, EvalSummary
__all__ = ["RestoreResult", "restore_image", "evaluate_samples", "EvalSummary"]
# -*- coding: utf-8 -*-
"""单张图像复原：给定策略与工具箱，执行序列决策复原流程。"""
from __future__ import annotations
from dataclasses import dataclass
from typing import List, Optional
from PIL import Image
from ..agent.env import EnvConfig, RestorationEnv
from ..agent.policy import Policy
from ..agent.runner import run_episode
from ..agent.trajectory import Trajectory
from ..data.dataset import Sample
from ..data.synthesize import DegradationSpec
from ..tools.toolbox import Toolbox
@dataclass
class RestoreResult:
  """单图复原结果：复原图像、工具序列、复原日志与完整轨迹。"""
  image: Image.Image
  tool_sequence: List[str]
  restoration_log: List[str]
  num_turns: int
  finished: bool
  trajectory: Trajectory
def restore_image(image: Image.Image, policy: Policy, toolbox: Toolbox,
                  env_config: Optional[EnvConfig] = None,
                  degradation_hint: Optional[List[str]] = None) -> RestoreResult:
  """
  degradation_hint 为可选的退化类型提示；为 None 时环境不提供提示，由智能体自行推理
   （此时 env_config.provide_degradation_hint 被置为 False）。
  """
  cfg = env_config or EnvConfig()
```

```
    if degradation_hint is None:
        cfg.provide_degradation_hint = False
        specs: List[DegradationSpec] = []
    else:
        specs = [DegradationSpec(d, "unknown") for d in degradation_hint]
    sample = Sample(degraded=image.convert("RGB"), clean=None, degradations=specs)
    traj = run_episode(RestorationEnv(toolbox, cfg), policy, sample)
    return RestoreResult(
        image=traj.final_image, tool_sequence=traj.executed_tools, restoration_log=traj.restoration_log,
        num_turns=len(traj.steps), finished=traj.finished, trajectory=traj,
    )
# -*- coding: utf-8 -*-
"""
数据集评估：对每个样本运行复原流程，计算退化图/复原图相对真值的 PSNR、SSIM（可选 LPIPS）
与无参考质量分，统计工具调用次数与耗时，输出 CSV、汇总文本与图像。
"""
from __future__ import annotations
import json
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Dict, List, Optional, Sequence
import pandas as pd
from ..agent.env import EnvConfig, RestorationEnv
from ..agent.policy import Policy
from ..agent.runner import run_episode
from ..data.dataset import Sample
from ..reward.composite import RewardConfig, compute_episode_reward
from ..reward.quality_metrics import lpips_distance, no_reference_score, psnr, ssim
from ..tools.toolbox import Toolbox
@dataclass
class EvalSummary:
    """数据集评估汇总：平均指标与逐样本结果表。"""
    num_samples: int
    mean_psnr_degraded: float
    mean_psnr_restored: float
    mean_ssim_degraded: float
    mean_ssim_restored: float
    mean_no_ref_restored: float
    mean_tool_calls: float
    mean_seconds: float
    mean_reward: float
    table: pd.DataFrame = field(repr=False)
    def to_text(self) -> str:
        """生成汇总文本。"""
        return "\n".join([
            f"samples: {self.num_samples}",
            f"PSNR  degraded -> restored: {self.mean_psnr_degraded:.2f} -> {self.mean_psnr_restored:.2f}",
            f"SSIM  degraded -> restored: {self.mean_ssim_degraded:.4f} -> {self.mean_ssim_restored:.4f}",
            f"no-reference score (restored): {self.mean_no_ref_restored:.4f}",
```

```
            f"mean tool calls: {self.mean_tool_calls:.2f}",
            f"mean seconds per image: {self.mean_seconds:.2f}",
            f"mean episode reward: {self.mean_reward:.4f}",
        ])
def evaluate_samples(samples: Sequence[Sample], policy: Policy, toolbox: Toolbox,
                     env_config: Optional[EnvConfig] = None, reward_config: Optional[RewardConfig] = None,
                     output_dir: Optional[str | Path] = None, use_lpips: bool = False,
                     progress: Optional[Callable[[int, int], None]] = None) -> EvalSummary:
    """对样本集合运行完整复原流程并计算指标，可输出 CSV、汇总与图像。"""
    env_config = env_config or EnvConfig()
    reward_config = reward_config or RewardConfig()
    rows: List[Dict] = []
    out = Path(output_dir) if output_dir else None
    if out:
        (out / "images").mkdir(parents=True, exist_ok=True)
        (out / "details").mkdir(parents=True, exist_ok=True)
    for i, sample in enumerate(samples):
        env = RestorationEnv(toolbox, env_config)
        start = time.time()
        traj = run_episode(env, policy, sample)
        elapsed = time.time() - start
        breakdown = compute_episode_reward(traj, reward_config)
        row: Dict = {
            "sample_id": sample.index, "gt_degradations": ", ".join(sample.degradation_types),
            "tool_sequence": " -> ".join(traj.executed_tools), "restoration_log": ", ".join(traj.restoration_log
    ),
            "num_tool_calls": traj.num_tool_calls, "num_turns": len(traj.steps), "finished": traj.finished,
            "seconds": elapsed, "reward": breakdown.score, "format_score": breakdown.format_score,
            "no_ref_restored": no_reference_score(traj.final_image),
        }
        if sample.clean is not None:
            row.update({
                "psnr_degraded": psnr(sample.degraded, sample.clean), "psnr_restored": psnr(traj.final_image, sa
    mple.clean),
                "ssim_degraded": ssim(sample.degraded, sample.clean), "ssim_restored": ssim(traj.final_image, sa
    mple.clean),
            })
            if use_lpips:
                row["lpips_degraded"] = lpips_distance(sample.degraded, sample.clean)
                row["lpips_restored"] = lpips_distance(traj.final_image, sample.clean)
        rows.append(row)
        if out:
            d = out / "images" / f"sample_{sample.index:04d}"
            d.mkdir(exist_ok=True)
            sample.degraded.save(d / "degraded.png")
            traj.final_image.save(d / "restored.png")
            if sample.clean is not None:
                sample.clean.save(d / "gt.png")
            (out / "details" / f"sample_{sample.index:04d}.json").write_text(
                json.dumps({"responses": traj.responses, "row": row}, ensure_ascii=False, indent=2, default=str)
```

```
    ,
                encoding="utf-8")
        if progress:
            progress(i + 1, len(samples))
    table = pd.DataFrame(rows)
    has_ref = "psnr_restored" in table.columns and len(table) > 0
    summary = EvalSummary(
        num_samples=len(table),
        mean_psnr_degraded=float(table["psnr_degraded"].mean()) if has_ref else float("nan"),
        mean_psnr_restored=float(table["psnr_restored"].mean()) if has_ref else float("nan"),
        mean_ssim_degraded=float(table["ssim_degraded"].mean()) if has_ref else float("nan"),
        mean_ssim_restored=float(table["ssim_restored"].mean()) if has_ref else float("nan"),
        mean_no_ref_restored=float(table["no_ref_restored"].mean()) if len(table) else float("nan"),
        mean_tool_calls=float(table["num_tool_calls"].mean()) if len(table) else 0.0,
        mean_seconds=float(table["seconds"].mean()) if len(table) else 0.0,
        mean_reward=float(table["reward"].mean()) if len(table) else 0.0,
        table=table,
    )
    if out:
        table.to_csv(out / "evaluation_results.csv", index=False)
        (out / "summary.txt").write_text(summary.to_text(), encoding="utf-8")
    return summary
# -*- coding: utf-8 -*-
"""
命令行入口：restoramind <子命令>
    make-data   从干净图像目录合成复合退化数据集（Parquet）
    serve-tools 启动复原工具 HTTP 服务
    train       GRPO 训练（轻量表格化策略；大模型训练见 scripts/train_verl.sh）
    restore     单图复原
    evaluate    数据集评估
"""
from __future__ import annotations
import argparse
import sys
from pathlib import Path
from typing import List, Optional
from PIL import Image
from . import __software_name__, __version__
def _build_policy(args):
    from .agent.policy import HeuristicPolicy, OpenAICompatiblePolicy
    if args.policy == "openai":
        return OpenAICompatiblePolicy(base_url=args.api_url, api_key=args.api_key, model=args.model)
    if args.policy == "softmax":
        from .train.softmax_policy import SoftmaxToolPolicy
        if not args.policy_file:
            raise SystemExit("--policy softmax 需要同时指定 --policy-file（train 子命令产出的 policy.json）")
        policy = SoftmaxToolPolicy(seed=0)
        policy.load(args.policy_file)
        return policy
    return HeuristicPolicy(mode=args.mode)
```

```
def _build_toolbox(args):
    from .tools.toolbox import build_toolbox
    return build_toolbox(args.toolbox)
def cmd_make_data(args) -> int:
    """子命令 make-data：从干净图像目录合成复合退化数据集。"""
    from .agent.prompts import build_system_prompt, build_first_user_prompt
    from .data.dataset import build_parquet_dataset, list_images
    images = list_images(args.clean_dir)
    if not images:
        print(f"未在 {args.clean_dir} 找到图像", file=sys.stderr)
        return 1
    hint_template = "<image>\nImage analysis indicates these degradation types (order unknown): {degradations}\n
    Select the FIRST tool to apply."
    n = build_parquet_dataset(images, args.output, build_system_prompt(args.mode), hint_template, seed=args.seed
    ,
                              clean_prob=args.clean_prob, max_size=args.max_size,
                              samples_per_image=args.samples_per_image)
    print(f"写入 {n} 个样本 -> {args.output}")
    return 0
def cmd_serve_tools(args) -> int:
    """子命令 serve-tools：启动复原工具 HTTP 服务。"""
    from .tool_server import create_app, load_backend
    app = create_app(backend=load_backend(args.backend), family=args.family)
    app.run(host=args.host, port=args.port)
    return 0
def cmd_train(args) -> int:
    """子命令 train：运行轻量策略的 GRPO 训练。"""
    from .agent.env import EnvConfig, RestorationEnv
    from .data.dataset import load_parquet_samples
    from .reward.composite import RewardConfig
    from .train.grpo import GRPOConfig, GRPOTrainer
    from .train.softmax_policy import SoftmaxToolPolicy
    toolbox = _build_toolbox(args)
    samples = load_parquet_samples(args.data, args.num_samples)
    policy = SoftmaxToolPolicy(lr=args.lr, seed=args.seed)
    trainer = GRPOTrainer(lambda: RestorationEnv(toolbox, EnvConfig(mode="single_tool_iterative", max_turns=args
    .max_turns)),
                          policy, GRPOConfig(group_size=args.group_size, epochs=args.epochs, reward=RewardConfig
    ()))
    trainer.fit(samples, batch_size=args.batch_size,
                on_step=lambda s: print(f"step {s.step}: reward={s.mean_reward:.4f} quality={s.mean_quality:.4f}
     "
                                        f"violations={s.format_violation_rate:.2f} tools={s.mean_tool_calls:.2f}
    "))
    policy.save(args.output)
    print(f"策略已保存 -> {args.output}")
    return 0
def cmd_restore(args) -> int:
    """子命令 restore：对单张图像执行复原。"""
    from .agent.env import EnvConfig
```

```
    from .inference.restore import restore_image
    policy = _build_policy(args)   # 先校验策略参数，再读取图像
    image = Image.open(args.input).convert("RGB")
    hint: Optional[List[str]] = [x.strip() for x in args.degradations.split(",") if x.strip()] if args.degradati
    ons else None
    result = restore_image(image, policy, _build_toolbox(args),
                           EnvConfig(mode=args.mode, max_turns=args.max_turns), hint)
    result.image.save(args.output)
    print(f"工具序列: {' -> '.join(result.tool_sequence) or '(无)'}")
    print(f"复原结果 -> {args.output}")
    return 0
def cmd_evaluate(args) -> int:
    """子命令 evaluate：对数据集执行评估并输出结果。"""
    from .agent.env import EnvConfig
    from .data.dataset import load_parquet_samples
    from .inference.evaluate import evaluate_samples
    samples = load_parquet_samples(args.data, args.num_samples)
    summary = evaluate_samples(samples, _build_policy(args), _build_toolbox(args),
                               EnvConfig(mode=args.mode, max_turns=args.max_turns,
                                         provide_degradation_hint=not args.no_hint), output_dir=args.output)
    print(summary.to_text())
    return 0
def _common(p: argparse.ArgumentParser) -> None:
    p.add_argument("--mode", default="single_tool_iterative", choices=["single_tool_iterative", "multi_tool_plan
    ning"])
    p.add_argument("--toolbox", default="local", choices=["local", "remote"], help="local=经典算子；remote=远程
    专家模型服务")
    p.add_argument("--policy", default="heuristic", choices=["heuristic", "softmax", "openai"],
                   help="heuristic=规则策略；softmax=train 产出的轻量策略（需 --policy-file）；openai=vLLM 部署
    的多模态智能体")
    p.add_argument("--policy-file", default=None, help="softmax 策略参数文件（policy.json）")
    p.add_argument("--api-url", default="http://localhost:8000/v1")
    p.add_argument("--api-key", default="EMPTY")
    p.add_argument("--model", default=None)
    p.add_argument("--max-turns", type=int, default=6)
def build_parser() -> argparse.ArgumentParser:
    """构建命令行参数解析器。"""
    parser = argparse.ArgumentParser(prog="restoramind", description=f"{__software_name__} V{__version__}")
    parser.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    sub = parser.add_subparsers(dest="command", required=True)
    p = sub.add_parser("make-data", help="合成复合退化数据集")
    p.add_argument("--clean-dir", required=True)
    p.add_argument("--output", required=True)
    p.add_argument("--mode", default="single_tool_iterative", choices=["single_tool_iterative", "multi_tool_plan
    ning"])
    p.add_argument("--seed", type=int, default=0)
    p.add_argument("--clean-prob", type=float, default=0.0)
    p.add_argument("--max-size", type=int, default=512)
    p.add_argument("--samples-per-image", type=int, default=1)
    p.set_defaults(func=cmd_make_data)
```

```
    p = sub.add_parser("serve-tools", help="启动复原工具服务")
    p.add_argument("--family", default=None, help="仅服务某个模型家族，如 SCUNet")
    p.add_argument("--backend", default="classical", help="classical 或 module:function")
    p.add_argument("--host", default="0.0.0.0")
    p.add_argument("--port", type=int, default=5000)
    p.set_defaults(func=cmd_serve_tools)
    p = sub.add_parser("train", help="GRPO 训练（轻量策略）")
    _common(p)
    p.add_argument("--data", required=True)
    p.add_argument("--output", default="policy.json")
    p.add_argument("--num-samples", type=int, default=None)
    p.add_argument("--epochs", type=int, default=1)
    p.add_argument("--batch-size", type=int, default=4)
    p.add_argument("--group-size", type=int, default=4)
    p.add_argument("--lr", type=float, default=0.5)
    p.add_argument("--seed", type=int, default=0)
    p.set_defaults(func=cmd_train)
    p = sub.add_parser("restore", help="单图复原")
    _common(p)
    p.add_argument("--input", required=True)
    p.add_argument("--output", required=True)
    p.add_argument("--degradations", default=None, help="可选的退化类型提示，逗号分隔")
    p.set_defaults(func=cmd_restore)
    p = sub.add_parser("evaluate", help="数据集评估")
    _common(p)
    p.add_argument("--data", required=True)
    p.add_argument("--output", required=True)
    p.add_argument("--num-samples", type=int, default=None)
    p.add_argument("--no-hint", action="store_true",
                   help="不向智能体提供真实退化类型提示（检验自行诊断能力；规则策略依赖提示，此时会直接作答）")
    p.set_defaults(func=cmd_evaluate)
    return parser
def main(argv: Optional[List[str]] = None) -> int:
    """命令行入口，返回进程退出码。"""
    args = build_parser().parse_args(argv)
    return args.func(args)
if __name__ == "__main__":
    raise SystemExit(main())
# -*- coding: utf-8 -*-
from .cli import main
raise SystemExit(main())
```